PATHFINDING · 游戏客户端秋招版

A* 寻路：从原理到工程

从 BFS、Dijkstra 一路推到 A*：f = g + h 为什么能保证最优，启发函数怎么选，open list 怎么实现才快；再往下是 JPS、分层寻路、导航网格、分帧寻路、帧同步确定性这些游戏里真正要面对的问题。

6 种搜索算法 | 5 个交互实验（可画墙、拖动起终点） | MSVC /O2 实测 10 组实验 | 40 道面试追问

CHAPTER 00

先背这个：30 秒面试版答案

面试官问“讲讲 A*”，先用一分钟讲清全貌，再等他挑一个点深挖。

问：讲讲 A* 算法。

A* 是带启发信息的最短路搜索。它给每个节点算一个 `f(n) = g(n) + h(n)`：g 是从起点走到 n 的真实代价，h 是从 n 到终点的**估计**代价。每次从 open list 里取 f 最小的节点展开，更新邻居的 g，直到取出终点。

- **它是 Dijkstra 和贪心的折中**：h ≡ 0 就退化成 Dijkstra（只看走了多远，向四周均匀扩散）；只看 h 就是贪心最佳优先（直奔终点，快但不保证最短）。
- **最优性的前提是 h 可采纳（admissible）**：h 永远不高估真实剩余代价。如果再满足**一致性**（h(n) ≤ cost(n, n') + h(n')），节点第一次出 open 时 g 就已经最优，closed 节点不用重开。
- **h 要和移动规则匹配**：4 方向用曼哈顿，8 方向用 Octile（对角距离），任意角度用欧几里得。h 越接近真实值，展开的节点越少；高估就会丢掉最优性，但能换来速度（加权 A*）。

工程上：open list 用二叉堆，配合“重复入队 + 出队时丢弃过期条目”的惰性删除；节点状态用整图数组 + generation 计数复位，不要每次查询清零；f 相同时优先展开 h 小的节点（平局决胜）。

放到游戏里：大地图用导航网格（Recast/Detour：多边形上做 A*，再用漏斗算法拉直路径）或分层寻路；格子图可以用 JPS 加速；长路径要分帧做；先用连通块编号判断终点是否可达；帧同步游戏里代价用整数，比较器用 (f, h, 节点编号) 的全序，否则各端走出的路径会不一样。

家族速查表

算法	取出顺序	最优?	搜索范围	一句话特点
BFS	先进先出（按步数）	只在**等权图**上最优	以起点为圆心向外扩	不看代价：8 方向时斜走也只算 1 步
Dijkstra	g 最小	✓	以起点为圆心按代价扩	一次能求出到所有点的最短路
贪心最佳优先	h 最小	✗	朝终点直冲，碰壁才绕	开阔地极快，遇到凹形障碍会钻进去
A*	g + h 最小	✓ **（h 可采纳）**	朝终点的“椭圆”	h 越准搜得越少；h = 0 就是 Dijkstra
加权 A*	g + w·h 最小	**代价 ≤ w × 最优**	介于 A* 与贪心之间	用有上界的次优换速度
JPS	g + h 最小（只对跳点）	✓	沿直线“跳”，只把拐点入堆	只适用于均匀代价的格子图

面试注意：最容易丢分的五句话

- “A* 一定比 Dijkstra 快”——**不一定。**启发函数被障碍误导时，A* 也会搜很大一片（本书 E1：迷宫里 A* 展开 79476 个节点，Dijkstra 是 80180 个，几乎一样）；目标不可达时两者都要把整个连通块搜完。
- “h 越大越好”——**错。**h 一旦超过真实代价就不再保证最短（E2：8 方向地图用曼哈顿距离，随机障碍图上 40 条路径没有一条最短，最多比最短路长 9.8%）。
- “8 方向用欧几里得距离最准”——**不对。**8 方向下的真实最短距离是 Octile，欧几里得偏小，搜得更多（E2：16105 对 11758 个节点）。
- “open list 用 std::set 方便改 key”——能用，但比二叉堆慢（E5：2.21 ms 对 1.30 ms）。**最常见的做法是允许重复入队，出队时丢弃过期条目。**
- “寻路结果只要代价对就行”——在帧同步游戏里不行。只比 f 的话，两份堆实现给出的路径 40 次里 40 次不同（E9），各端单位走的路不一样，状态就会不同步。

本书怎么读

第一部分 · 图搜索家族

寻路问题怎么建模；BFS → Dijkstra → 贪心 → A* 是怎么一步步演化出来的。

第二部分 · A* 核心

逐步图解、伪代码、C++；启发函数的可采纳性与一致性；最优性证明与复杂度。

第三部分 · 工程实现与加速

open list 和节点状态的数据结构；JPS、HPA*、双向搜索、路径平滑。

第四部分 · 游戏实战

导航网格与漏斗算法、分帧寻路、动态障碍、流场、帧同步确定性；实测总表与 40 道面试题。

关于实测数据

书中所有“实测”都来自 `demos/astar_demo.cpp`，在 Windows 11 + MSVC 14.44（`cl /std:c++latest /EHsc /O2`，x64）下运行，完整输出见 `demos/run_output.txt`。统一条件：**512×512 格子图，8 方向移动，禁止切角，直走代价 1000、斜走 1414（整数）**；五种地图：开阔地、随机障碍 25%、U 型陷阱、迷宫、房间；每张图 40 次随机查询取平均。绝对耗时与机器有关，**请看相对关系。**书中的步骤图由构建脚本真实运行搜索后生成（直走 10、斜走 14），格子里的数字都是算出来的。

PART ONE

第一部分　从 BFS 到 A*：图搜索家族

A* 不是凭空出现的。把 BFS、Dijkstra、贪心最佳优先放在一起比较，就能看清 A* 的 f = g + h 到底组合了什么。

CHAPTER 01

寻路问题的建模，以及 A* 的四位前辈

先把“地图”变成“图”，再看四种算法各自按什么顺序访问节点。

本章目标

- 说清游戏里三种常见的寻路图：格子图、路点图、导航网格
- 理解 4 / 8 方向、移动代价和“禁止切角”这几条移动规则
- 看懂 BFS、Dijkstra、贪心最佳优先在同一张图上的搜索范围差异
- 能用一句话说出 A* 和它们的关系

1.1 把地图变成图

寻路的本质是在带权图 G = (V, E, cost) 上求起点到终点代价最小的路径。游戏地图本身不是图，要先选一种表示方法：

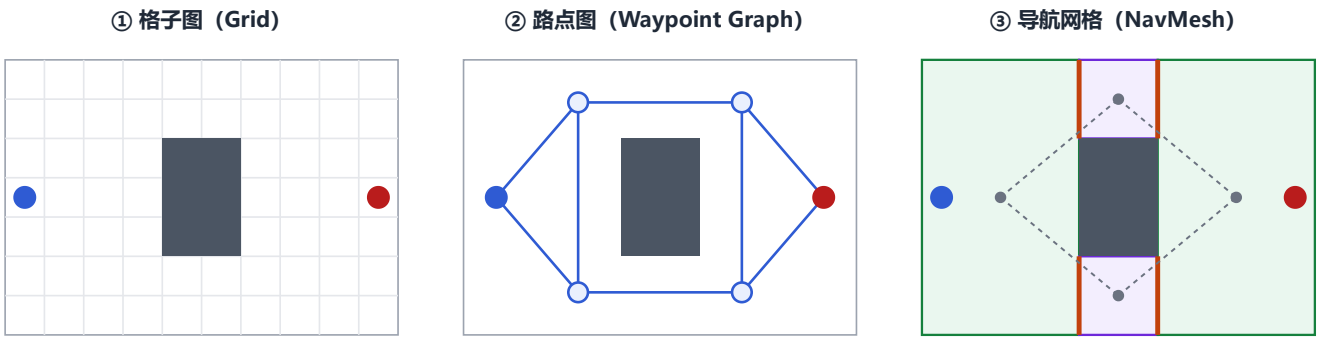


图 1-1 游戏里的三种寻路图：A* 在三者上都能用，区别只在于“节点是什么、邻居怎么取、代价怎么算”

表示	优点	缺点	典型场景
格子图	实现最简单；动态障碍改一个格子就行；天然适合 JPS、流场	节点多、内存大；路径是锯齿状的，要做平滑	RTS、塔防、战棋、2D 游戏、服务器上的粗略寻路
路点图	节点很少，速度最快	只能沿边走；美术改地图就要重摆路点；覆盖不全时会“走不到”	早期 FPS、固定路线的 NPC、MOBA 小兵沿兵线走
导航网格	节点少，又能覆盖整片可走区域；路径自然	生成复杂（体素化、区域划分、三角化）；动态修改代价高	3D 游戏的主流：UE、Unity 内置寻路，Recast/Detour

后面的算法讲解统一用格子图，因为它最直观；第 7 章再回到导航网格。**A* 本身和图的表示无关**：只要能回答“这个节点有哪些邻居、走过去花多少、离终点估计还有多远”，就能跑 A*。

1.2 移动规则：方向、代价与“切角”

- **4 方向**：只能上下左右，每步代价 1。
- **8 方向**：再加 4 个斜向。斜走代价应为 $\sqrt{2} \approx 1.414$；如果也算 1，路径会莫名其妙地多走斜线。工程上常用整数 10 / 14 或 1000 / 1414，避免浮点误差。
- **禁止切角**：斜走时，两侧的直邻格都必须可走。否则角色会贴着墙角“穿”过去，在 3D 里表现为模型卡进墙里。

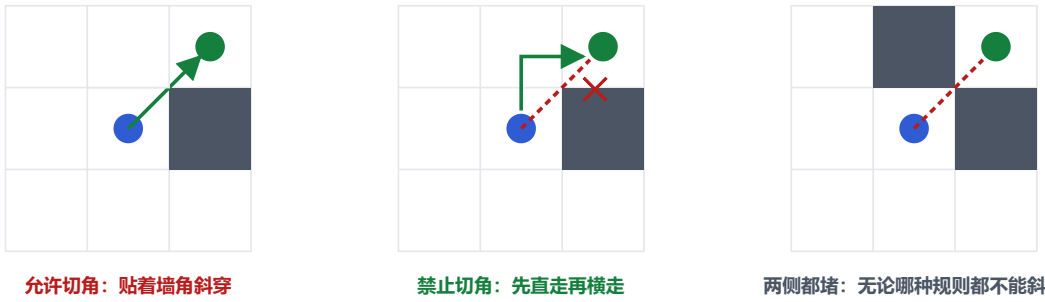


图 1-2 “切角”规则。本书统一禁止切角；这时“8 方向可达”和“4 连通”是等价的，判断连通性可以用简单的 4 连通 flood fill

面试注意：切角是一个常见的线上 bug

“小兵卡在墙角”“角色贴墙穿过去了”，很多时候就是寻路允许切角，而物理碰撞不允许。这时寻路给出的路径物理上走不通，角色会卡住或抖动。**寻路的移动规则必须和角色真实的碰撞体积一致**，导航网格要按角色半径向内收缩（agent radius），也是这个道理。

1.3 四位前辈：按什么顺序访问节点

所有图搜索都有同一个骨架：维护一个“待处理”集合（open / frontier），每次取出一个节点展开，把它的邻居放进去。**区别只在于“每次取哪一个”**：

算法	open 的数据结构	每次取出	看了什么信息
BFS	队列（FIFO）	最早放进去的	只看步数
Dijkstra	优先队列	g 最小的（离起点代价最小）	只看“已经走了多远”
贪心最佳优先	优先队列	h 最小的（估计离终点最近）	只看“估计还要走多远”
A*	优先队列	**f = g + h 最小的**	两者都看：经过它的整条路径估计有多长

下面四张图是在同一张带 U 型障碍的地图上，四种算法跑完后的状态。蓝色是已经展开过的节点（closed），绿色是还在 open 里等待的节点，橙色是最终路径。标题里的“代价”用的是直走 10、斜走 14。

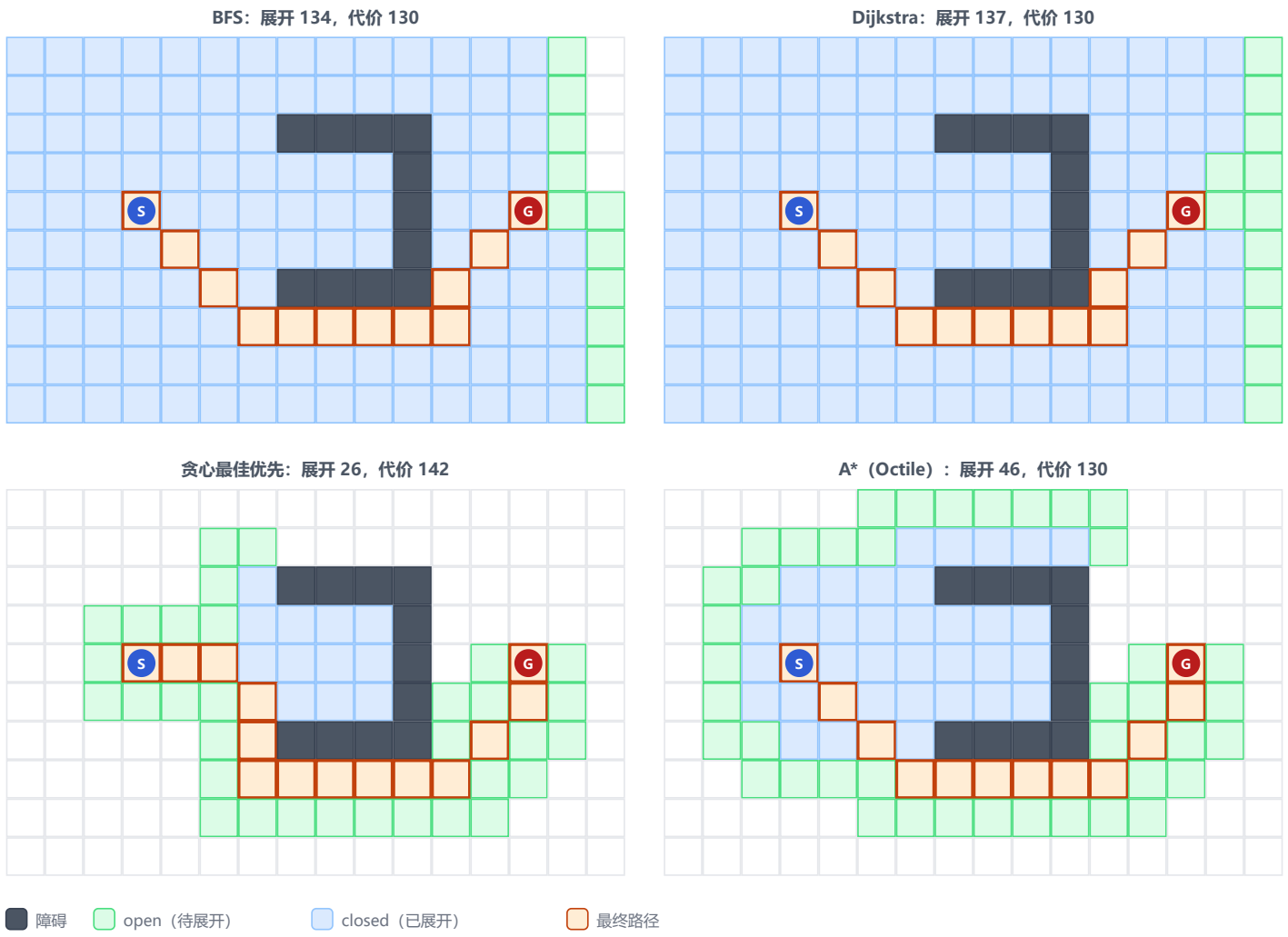


图 1-3 同一张图上的四种搜索：BFS 和 Dijkstra 向四周扩散；贪心钻进 U 型凹口再绕出来，路径更长；A* 搜得比 Dijkstra 少，路径同样最优

BFS：只在等权图上最优

BFS 一层一层往外扩，第一次碰到终点时走的**步数**最少。但 8 方向时斜走代价是 1.414，步数最少不等于代价最小。实测 E1 中，随机障碍图上 BFS 有 72% 的路径不是最短的。BFS 的价值在于简单、常数小：在等权图上（比如 4 方向、每步代价相同的战棋），它就是最好的选择；它还常用来做 flood fill、计算连通块、计算“几步之内能走到哪”（战棋的移动范围）。

Dijkstra：按 g 扩散

Dijkstra（1959）每次取出 g 最小的节点。边权非负时，一个节点第一次被取出，它的 g 就是最短距离。它的搜索范围是以起点为圆心的“等代价圈”，**完全不知道终点在哪个方向**，所以要把离起点比终点更近的点全部展开一遍。

Dijkstra · 伪代码

```
dist[S] = 0；把 S 放入优先队列
while 队列非空：
    u = 取出 dist 最小的节点
    if u 已处理：continue
    标记 u 已处理
    if u == G：break                    // 单目标时可以提前结束
    for 每条边 (u, v, w)：
        if dist[u] + w < dist[v]：
            dist[v] = dist[u] + w；parent[v] = u；把 v 放入队列
```

它的优点是**一次搜索能得到起点到所有点的最短路**。游戏里如果要“从一个点出发，算到很多目标的距离”（比如 AI 要从所有资源点里挑最近的一个，或者计算整张图的距离场），一次 Dijkstra 比跑很多次 A* 划算得多。

贪心最佳优先：按 h 直冲

只看 h，每次都走“看起来离终点最近”的节点。开阔地上它快得惊人（E1：开阔地展开 228 个节点，Dijkstra 要 127329 个），但它对已经走了多远毫不在意，会钻进凹形障碍再绕出来，路径也不保证最短（E1：随机障碍图上 100% 非最优，平均长 14%）。

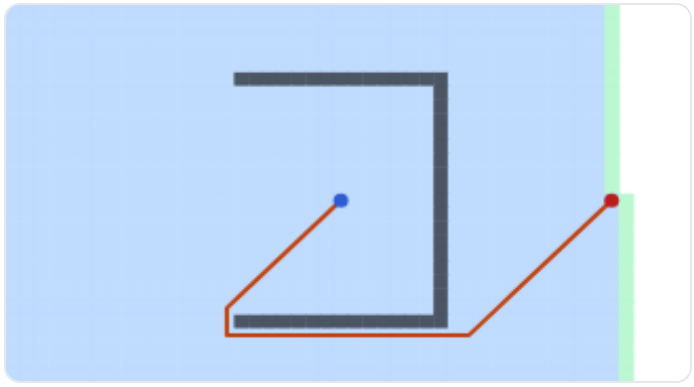
A*：把两者加起来

Hart、Nilsson、Raphael 在 1968 年提出 A*。f(n) = g(n) + h(n) 估计的是“经过 n 的那条完整路径有多长”。g 保证它不会像贪心那样无视已走过的代价，h 让它像贪心一样朝终点偏。只要 h 不高估，A* 就和 Dijkstra 一样能找到最短路，同时搜得更少。

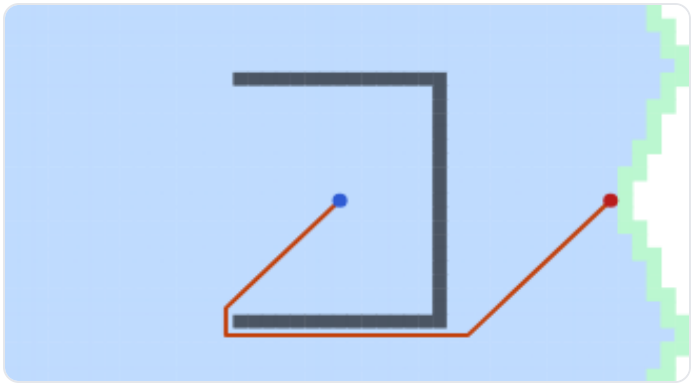
1.4 交互实验：同一张图，四种算法

切换地图试试：在“U 型陷阱”上贪心的路径明显绕远；在“迷宫”里 A* 和 Dijkstra 几乎一样，因为启发函数被墙误导了；“空地”上 A* 只展开路径附近的格子。

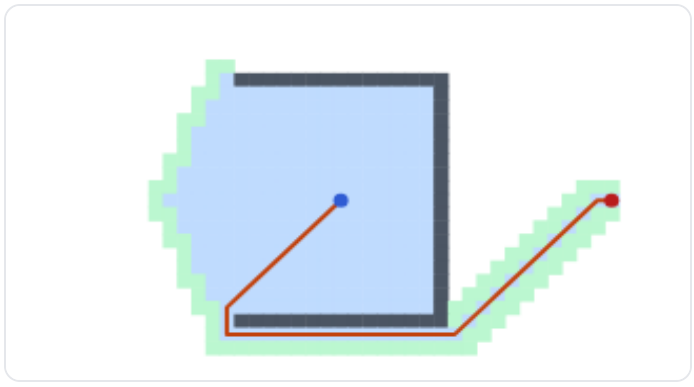
交互实验 · 同一张图，不同算法搜了多大范围



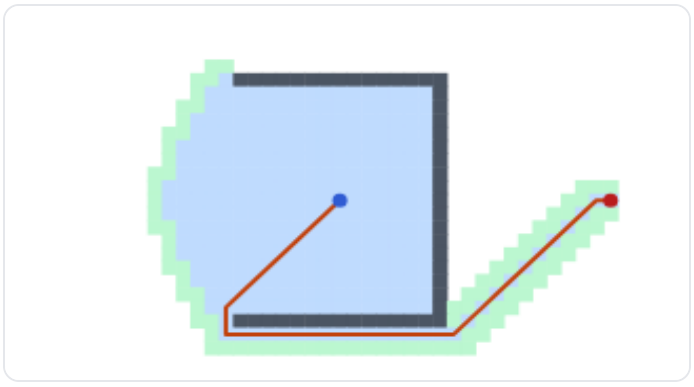
BFS 展开 **1143** 代价 44.46（最优）



Dijkstra 展开 **1211** 代价 44.46（最优）



贪心最佳优先 • Octile（八方向） 展开 **317** 代价 44.46（最优）



A* • Octile（八方向） 展开 **327** 代价 44.46（最优）

open（待展开） closed（已展开） 当前节点 JPS 扫描过 路径 障碍

实测 E1（节选）：512×512，40 次查询的平均展开节点数

地图	BFS	Dijkstra	贪心	A*
开阔地	128450	127329	228	**228**
随机障碍 25%	91515	90947	360（非最优）	**11758**
U 型陷阱	205173	207016	42496（52% 非最优）	**52359**
迷宫	80179	80180	55842	**79476**
房间	114689	115138	382（98% 非最优）	**7822**

本章速记

- 寻路 = 带权图上的最短路。三种图：**格子**（简单、动态好改）、**路点**（最少节点）、**导航网格**（3D 主流）。
- 8 方向斜走代价 √2（整数 14 / 1414）；**禁止切角**，寻路规则必须和碰撞体积一致。
- 四种算法只差“每次取出谁”：BFS 取最早的，Dijkstra 取 g 最小，贪心取 h 最小，**A* 取 g + h 最小。**
- A* 在开阔地上和贪心一样快，又和 Dijkstra 一样最优；但启发函数被障碍误导时（迷宫），它也会退化到接近 Dijkstra。

▶ **自测：战棋游戏里要显示“这个单位这回合能走到哪些格子”，用什么算法?**

▶ **自测：要从 30 个补给点里找离玩家最近的一个，跑 30 次 A* 合适吗?**

PART TWO

第二部分　A* 核心：过程、启发函数、正确性

这一部分回答三个问题：A* 每一步具体在做什么；h 应该怎么设计；为什么在 h 不高估的前提下，A* 找到的一定是最短路。

CHAPTER 02

A* 逐步拆解

open、closed、g、h、f、parent，六个概念，一个循环。

本章目标

- 准确说出 g、h、f 和 open / closed 的含义
- 能跟着一张 7×5 的小地图手算前几步，理解 f 为什么会逐步上升
- 写出 5 分钟能写完、没有经典 bug 的面试版 A*

2.1 六个概念

符号	含义	谁来维护
`g(n)`	从起点到 n **目前找到的**最小代价，是真实走出来的	每次发现更短的路就更新
`h(n)`	从 n 到终点代价的**估计**，由启发函数给出，不依赖搜索过程	第一次见到 n 时计算一次
`f(n)`	`g(n) + h(n)`：经过 n 的整条路径的估计长度	g 变化时一起变
`parent(n)`	目前最短路径上 n 的前一个节点	与 g 同时更新；最后沿它回溯出路径
open	已发现、还没展开的节点（搜索的“边界”）	优先队列，按 f 取最小
closed	已展开、g 已确定为最优的节点	布尔数组或标记

2.2 算法主循环

A* · 伪代码

```
A*(S, G):
    g[S] = 0；parent[S] = 空
    open = { S }，按 f = g + h 排序
    while open 非空:
        cur = open 中 f 最小的节点；从 open 移到 closed
        if cur == G:                          // ① 出队时才判断终点
            return 沿 parent 从 G 回溯到 S 的路径
        for nb in cur 的每个邻居:
            if nb 是障碍 或 nb 在 closed: continue
            ng = g[cur] + cost(cur, nb)
            if nb 不在 open 或 ng < g[nb]:   // ② 发现更短的路才更新
                g[nb] = ng；parent[nb] = cur
                f[nb] = ng + h(nb)
                把 nb 放入 open（已在 open 中则更新它的 f）
    return 无路径                             // ③ open 空了还没取出 G
```

三个标注点就是面试里最常被追问的细节：

1. **为什么在出队时判断终点，而不是在入队时？** G 第一次入队时的 g 可能来自一条较长的路径，之后还有机会被更短的路径更新。只有当 G 成为 open 里 f 最小的节点时，才能确定没有更短的路了（第 4 章证明）。
2. **为什么要比较 ng < g[nb]？** 同一个节点可以从多个方向被发现，只保留最短的那条。忘记这个判断会让 parent 被较差的路径覆盖。
3. **open 空了意味着什么？** 起点所在连通块已经搜完了，终点不可达。这是 A* 的最坏情况，第 4 章会专门讨论。

2.3 手算一遍：7×5 地图，中间一堵墙

8 方向、禁止切角，直走 10、斜走 14，启发函数用 Octile 距离 h = 10·max(dx, dy) + 4·min(dx, dy)。S 在 (1, 2)，G 在 (5, 2)，中间 x = 3 一列有三格墙。下图每个格子里：**中间大字是 f，左下蓝字是 g，右下橙字是 h**；小箭头指向 parent。



图 2-1　A* 在小地图上的完整过程（构建脚本真实运行生成）。Dijkstra 在这张图上要展开 30 个节点，A* 只展开 14 个

逐步解读

- **第 1 步**：S 的 g = 0，h = 40（离 G 横向 4 格），f = 40。展开后 8 个邻居入 open。正右方的 (2, 2) g = 10，h = 30，f = 40；斜向的 (2, 1) g = 14，h = 34，f = 48；左侧的 (0, 2) g = 10，h = 50，f = 60。**朝着终点的方向 f 小，背离终点的方向 f 大**，这就是 h 起的作用。
- **第 2 步**：取出 f = 40 的 (2, 2)。它右边就是墙，只能发现上下几个格子，它们都已经在 open 里了，而且经过 (2, 2) 并不会更短，所以不更新。
- **第 3～4 步**：open 里最小的 f 变成 48，是 (2, 1) 和 (2, 3)。它们的 f 和 h 都相同，谁先出队取决于平局决胜规则（这里按入队顺序）。注意 **f 从 40 涨到了 48**：墙挡住了直线，估计值 h 太乐观了，现实把 f 抬了上去。

- **之后**：f = 54、60、62……依次被展开，直到绕过墙的那条路的 f 达到 68，并取出 G。最终 f(G) = g(G) = 68：**终点的 h = 0，所以它的 f 就是真实代价。**

> **深入一层：被展开节点的 f 永远不减**
>
> 回看第 1 ~ 4 步取出的 f：40, 40, 48, 48, 54, 54, 60, 62……从不下降。这不是巧合：当 h 满足一致性（第 3 章）时，沿着任何一条边走，f 都不会变小，所以 A* 是按 f 从小到大“一圈一圈”地展开节点，和 Dijkstra 按 g 一圈一圈展开完全对应。**A* 就是在“修改过边权”的图上跑 Dijkstra**，修改后的边权是 cost(u, v) − h(u) + h(v)。

同一张图上，最终路径可以从墙的上方或下方绕过，两条都是 68。上图走的是下方，而第 2.4 节的面试版代码在同一张图上走的是上方（`demos/interview_output.txt`）。**代价相同时走哪条，完全取决于平局时谁先出队**，这一点在第 7 章的帧同步部分会变得很重要。

2.4 面试手写版 C++

下面这份代码可以在 5 分钟内写完，覆盖了所有常见细节。它已在 MSVC 下编译通过，并用上面的地图测试过（`demos/astar_interview.cpp`）。

astar_interview.cpp（面试版：二叉堆 + 惰性删除 + 禁止切角）

```cpp
struct Grid {
    int w, h;
    std::vector<uint8_t> blocked;
    bool walkable(int x, int y) const { return x ≥ 0 && y ≥ 0 && x < w && y < h && !blocked[
};

struct Node { int f, g, idx; };
struct Cmp {   // priority_queue 是大顶堆：返回 true 表示 a 的优先级更低
    bool operator()(const Node& a, const Node& b) const { return a.f ≠ b.f ? a.f > b.f : a.g
};

std::vector<int> astar(const Grid& grid, int sx, int sy, int tx, int ty) {
    const int W = grid.w, H = grid.h;
    auto id = [W](int x, int y) { return y * W + x; };
    auto h = [&](int x, int y) {                                    // Octile：8 方向、直走 10、斜走
        int dx = std::abs(x - tx), dy = std::abs(y - ty);
        return 10 * std::max(dx, dy) + 4 * std::min(dx, dy);
    };
    std::vector<int> g(W * H, INT_MAX), parent(W * H, -1);
    std::vector<char> closed(W * H, 0);
    std::priority_queue<Node, std::vector<Node>, Cmp> open;
    g[id(sx, sy)] = 0;
    open.push({h(sx, sy), 0, id(sx, sy)});
    static const int DX[8] = {1, -1, 0, 0, 1, 1, -1, -1}, DY[8] = {0, 0, 1, -1, 1, -1, 1, -1};
    while (!open.empty()) {
        Node cur = open.top(); open.pop();
        if (closed[cur.idx]) continue;                              // 过期条目：该节点已用更小的 g 展
        closed[cur.idx] = 1;
        if (cur.idx == id(tx, ty)) break;                           // 出队时才判断终点
        int x = cur.idx % W, y = cur.idx / W;
        for (int d = 0; d < 8; ++d) {
            int nx = x + DX[d], ny = y + DY[d];
            if (!grid.walkable(nx, ny)) continue;
            if (d ≥ 4 && (!grid.walkable(nx, y) || !grid.walkable(x, ny))) continue;   // 禁止
            int j = id(nx, ny);
            if (closed[j]) continue;
            int ng = g[cur.idx] + (d < 4 ? 10 : 14);
            if (ng ≥ g[j]) continue;                                // 没有变短：忽略
            g[j] = ng; parent[j] = cur.idx;
            open.push({ng + h(nx, ny), ng, j});                     // 允许重复入队（惰性删除）
        }
    }
    std::vector<int> path;
```

```
    if (g[id(tx, ty)] == INT_MAX) return path;                  // 不可达
    for (int i = id(tx, ty); i != -1; i = parent[i]) path.push_back(i);
    std::reverse(path.begin(), path.end());
    return path;
}
```

demos/interview_output.txt

```
路径 7 个格子：(1,2) (2,1) (2,0) (3,0) (4,0) (5,1) (5,2)
  ..***..
  ..*#.*.
  .S.#.G.
  ...#...
  .......
封死以后：路径 0 个格子（空 = 不可达）
```

写完要主动说明的几个设计选择

- **open 允许重复入队**：`std::priority_queue` 不支持修改元素的 key。发现更短的 g 时直接再 push 一份，旧的那份出队时发现 `closed` 已置位就跳过。这叫惰性删除（lazy deletion），代价是堆里会有一些过期条目。
- **整数代价**：10 / 14 避免了浮点比较误差，平局判断是精确的。
- **平局决胜**：f 相同时优先 g 大（即 h 小）的节点，也就是更靠近终点的那个。开阔地上这个小改动能让展开节点数差一个数量级（E4：开阔地 3582 对 228）。
- **这个版本每次调用都分配整图数组**：面试足够了，但游戏里每帧可能有很多次短查询，要换成第 5 章的 generation 复位（E5：短查询 504 us 对 5.6 us）。

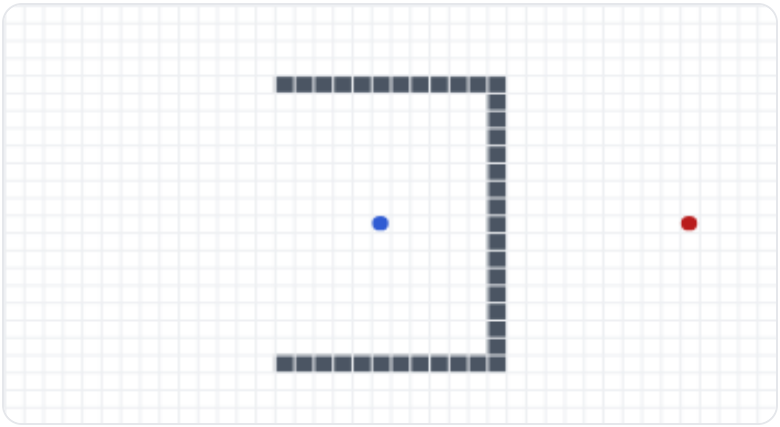
2.5 八个经典 bug

#	写法	后果
1	邻居**入队时**就判断是否为终点并返回	返回的不一定是最短路
2	出队后不检查 closed（惰性删除版）	同一节点被展开多次，结果仍对但变慢；若 parent 被过期条目覆盖，路径会错
3	发现更短的 g 时只改了 g 数组，没重新入队 / 更新堆	堆里还是旧的 f，展开顺序错误，可能得到非最短路
4	h 和移动规则不匹配（8 方向用曼哈顿）	h 高估，路径不是最短（E2：最多长 9.8%）
5	8 方向时斜走代价也写成 1（或 10）	路径莫名其妙地走锯齿斜线
6	允许切角，但角色的碰撞体积不允许	角色卡在墙角
7	用 `float` 做代价并用 `==` 比较 f	平局判断不稳定；不同平台结果不同
8	不处理不可达，或没有搜索上限	每次都把整个连通块搜完，卡一帧（E8：20.5 ms）

2.6 交互实验：寻路沙盒

默认是 U 型陷阱地图上的 A*。可以拖动 S、G，在空格上拖动画墙；右侧伪代码会高亮当前执行的行，鼠标悬停在格子上能看到 g、h、f。试试把算法切到 Dijkstra 和贪心，再切回 A*，对比 open（绿）和 closed（蓝）的形状。

交互实验 · 寻路沙盒（可拖动 S / G，在空格上拖动画墙、在墙上拖动擦除）



```
open = {S}；g[S] = 0
while open 非空：
  cur = open 中 f 最小的节点（移入 closed）
  if cur == G：沿 parent 回溯得到路径
  for nb in cur 的邻居（障碍 / closed 跳过）：
    ng = g[cur] + cost(cur, nb)
    if nb 不在 open 或 ng < g[nb]：
      g[nb] = ng；parent[nb] = cur
      f[nb] = g + h；放入 / 更新 open
return 无路径
```

点击「播放」或「单步」开始；鼠标悬停在格子上可查看 g / h / f。

open（待展开） closed（已展开） 当前节点 JPS 扫描过 路径 障碍

本章速记

- **f = g + h**：g 真实已走代价，h 估计剩余代价；每次展开 open 里 f 最小的节点。
- 三个细节：**出队时判终点**；**ng < g[nb] 才更新**；open 空即不可达。
- 被展开节点的 f 单调不减；终点的 f = g = 真实最短代价。
- 工程写法：priority_queue + 惰性删除、整数代价、f 相同取 g 大。

▶ **自测：A* 找到的路径代价是 68，那么搜索过程中被展开的节点，f 的最大值是多少?**

▶ **自测：closed 集合能不能不要?**

CHAPTER 03

启发函数：A* 的灵魂

同一个 A*，换一个 h，展开的节点数可以差 25 倍，结果也可能从最短变成不是最短。

本章目标

- 准确区分可采纳（admissible）与一致（consistent），知道两者分别保证了什么
- 能根据移动规则选出正确的距离公式，并说出其他公式错在哪
- 理解加权 A* 的取舍，以及它的次优上界
- 知道平局决胜为什么能让开阔地上的展开数降一个数量级

3.1 两个性质

记 h*(n) 为从 n 到终点的**真实**最短代价。

可采纳（admissible）

对所有节点 n：**h(n) ≤ h*(n)**。

也就是“只会低估，不会高估”。它保证 A* 找到的路径是**最短的**。

一致（consistent / 单调）

对每条边 (n, n')：**h(n) ≤ cost(n, n') + h(n')**，且 h(G) = 0。

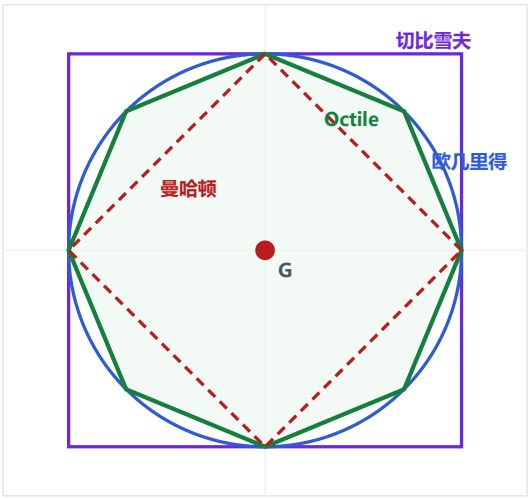
这是三角不等式。它保证节点**第一次出队时 g 已最优**，closed 节点永远不用重开。

- **一致 ⇒ 可采纳**：沿着从 n 到 G 的最短路径反复使用三角不等式，把每条边加起来，就得到 h(n) ≤ h*(n)。
- **可采纳 ⇏ 一致**：可以构造出“h(A) = 10，走一步代价 1 到 B，h(B) = 2”的例子，h 都不高估，但相邻两点的 h 落差比边长还大。这种 h 下，一个节点可能先以较大的 g 进入 closed，后来又发现更短的路，需要**重开**（reopen），否则就会丢失最优性。
- 实际使用的几种距离公式（曼哈顿、Octile、欧几里得等），只要和移动规则匹配，**都是一致的**。所以工程实现里通常直接不重开 closed 节点。

3.2 四种距离公式

设 $dx = |x - x_G|$，$dy = |y - y_G|$，直走代价 D，斜走代价 $D_2 = \sqrt{2} \cdot D$：

名称	公式	它是哪种移动的真实距离
曼哈顿	`D·(dx + dy)`	4 方向
Octile（对角距离）	`D·max(dx,dy) + (D₂ − D)·min(dx,dy)`	8 方向、斜走 √2（先斜走 min 步，再直走剩下的）
切比雪夫	`D·max(dx, dy)`	8 方向、斜走也算 D（国际象棋的王）
欧几里得	`D·√(dx² + dy²)`	任意角度移动（导航网格、连续空间）



图中每条线是“h = 12 格”的等值线

8 方向移动时，真实距离恰好是 12 格的点组成
绿色正八边形（Octile）。

某个启发函数的等值线如果在绿线**外面**，
说明绿线上的点 h 还不到 12 → **低估，可采纳**：
欧几里得（圆）、切比雪夫（正方形）

等值线在绿线**里面**，说明绿线上的点 h 超过 12
→ **高估，不可采纳**：曼哈顿（菱形）

离绿线越远，低估得越多，A* 要展开的节点也越多。

结论：启发函数要和移动规则匹配，贴着真实距离，又不越过它。8 方向就用 Octile。

图 3-1 四种距离的“单位圆”：Octile 恰好内接于欧几里得圆、外接于曼哈顿菱形

实测 E2：五种启发函数，同一批查询

启发函数	随机障碍 25%				房间		
	展开	耗时 ms	最短率	最多长出	展开	最短率	最多长出
h = 0（Dijkstra）	90947	6.26	100%	0	115138	100%	0
切比雪夫	22044	2.09	100%	0	27890	100%	0
欧几里得（取整）	16105	1.52	100%	0	16265	100%	0
Octile	**11758**	**1.39**	100%	0	**7822**	100%	0
曼哈顿（8 方向下高估）	3608	0.40	**0%**	**9.77%**	2577	**15%**	**8.39%**



图 3-2 可采纳的启发函数里，越接近真实距离展开越少：Octile 比欧几里得少一半，比 h = 0 少 93%；曼哈顿更少，但已经不保证最短

深入一层：整数化代价之后，欧几里得严格来说会高估

本书斜走代价取 1414，比 1000·√2 = 1414.21 略小。长对角线上，比如 100 格的纯对角路径：真实代价 141400，而欧几里得（取整）给出 141421，高估了 21。本次实测没有因此产生非最短路，但这个现象说明：**改了代价模型（取整、改斜走代价、加地形权重），就要重新检查启发函数是否还可采纳。**

Recast/Detour 的做法是把启发函数乘上 0.999（源码里的 `H_SCALE`），留一点余量防止浮点误差导致高估。

面试注意：地形代价会让距离公式“失真”

如果地图上有沼泽（代价 ×3）、道路（代价 ×0.5），那么启发函数必须乘以**最便宜地形**的单价，才能保证不高估。比如道路单价是 0.5，就得用 0.5·Octile。代价是：大部分区域不是道路，h 低估得很厉害，A* 的引导作用变弱。这是“地形越丰富，A* 越慢”的根本原因。

3.3 h 的强弱光谱



图 3-3 启发函数的光谱：h 越大，A* 越像贪心；h 越小，越像 Dijkstra。h 恰好等于真实距离时，A* 只展开最短路径上的节点

如果能拿到完美的启发函数 h = h*，A* 会径直沿着最短路径走，一个多余的节点都不展开。现实中拿不到 h*，但可以**预计算**出更接近它的下界，比如 ALT 算法预先算出到若干“地标”的距离，用三角不等式得到更紧的 h（第 6 章）。

3.4 加权 A*：用有上界的次优换速度

令 f = g + w·h（w > 1），h 被放大，搜索更“急”着冲向终点。可以证明，若 h 可采纳，加权 A* 找到的路径代价**不超过最优的 w 倍**。实践中远小于这个上界：

权重	随机障碍 25%				U 型陷阱		
	展开	耗时 ms	平均长出	最多长出	展开	平均长出	最多长出
w = 1.0（A*）	11758	1.341	0	0	52359	0	0
w = 1.2	**1159**	**0.112**	3.15%	7.26%	50077	0	0
w = 1.5	492	0.050	6.46%	12.52%	48430	0.17%	6.78%
w = 2.0	412	0.042	9.93%	16.10%	47323	0.76%	9.71%
w = 5.0	367	0.031	12.43%	21.95%	45720	5.52%	30.38%
贪心（w = ∞）	360	0.030	14.44%	27.56%	42496	10.55%	**58.05%**

- **随机障碍图**：w 只从 1 提到 1.2，展开数就降到 1/10，路径平均只长 3%。游戏里这通常是划算的：玩家看不出 3% 的绕路，却能感受到卡顿。
- **U 型陷阱**：加权几乎没用，展开数只从 52359 降到 45720。启发函数本来就把搜索引向杯底，加大它的权重只会让搜索更深地陷进去；必须先把杯子填满才能绕出来。
- **贪心没有上界**：U 型陷阱上最坏长出 58%。有上界的加权 A* 是更安全的选择。

变体

动态加权：离起点近时 w 大、离终点近时 w 趋于 1，开始冲得快，结尾走得准。**ARA*（Anytime Repairing A*）**：先用大 w 很快给出一条路，有余力再逐步减小 w 改进它，适合“先让单位动起来，路径后面再优化”的场景。

3.5 平局决胜：开阔地上的大问题

在没有障碍的开阔地上，从 S 到 G 有大量代价完全相同的最短路径，这些路径上的所有节点 f 都相等。如果 f 相同时随便选，A* 会把这一大片“平局区域”全部展开。规则很简单：**f 相同时，优先展开 h 更小（g 更大）的节点**，也就是离终点更近的那个，它会一路往前推，不再横向铺开。

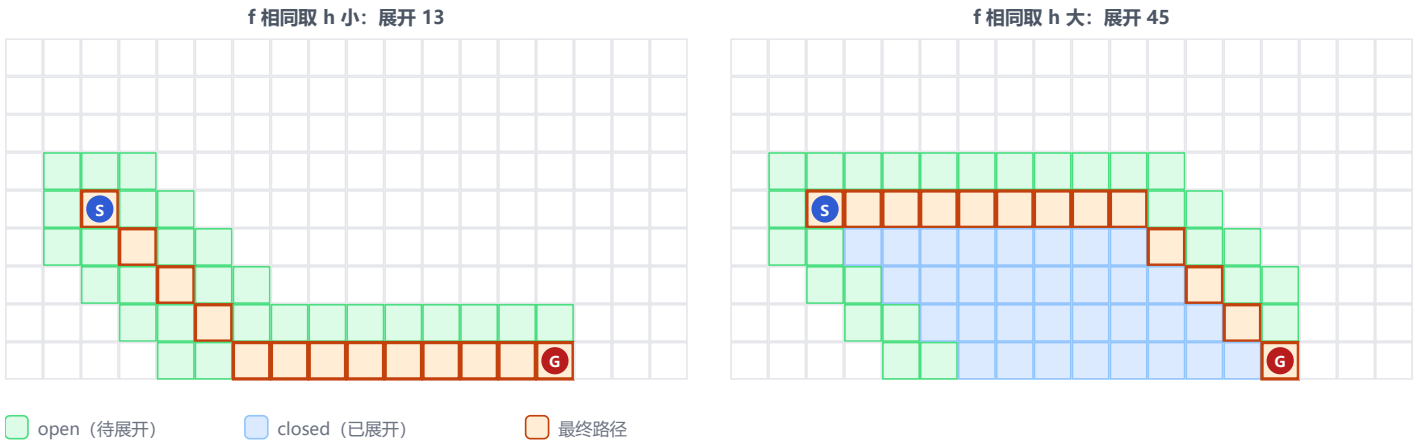


图 3-4　同一张空地图，两种平局规则：路径一样长（代价 136），展开数差 3 倍多；在 512×512 的大图上差距更大

E4：平均展开（耗时）	只比 f（交给堆）	f 同取 h 小	f 同取 h 大
开阔地	3582（0.292 ms）	**228（0.024 ms）**	**10772（0.517 ms）**
随机障碍 25%	11805	11758	11828
房间	8395	7822	8607

开阔地上差了 47 倍；障碍多的地图上平局本来就少，差别只有百分之几。另外两种常见的决胜方法：

- **微调 h**：h ×= (1 + 1/p)，p 取比最长路径还大的数。h 只放大一点点，几乎不影响最优性，但能打破平局。缺点是严格来说不再可采纳。
- **叉积决胜**：在 f 相同的节点里，优先选择更靠近“起点到终点连线”的节点，得到的路径看起来更直，不会先斜走到底再直走。

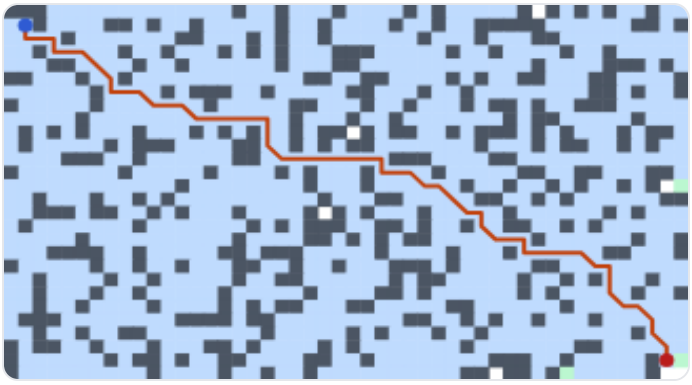
面试注意：平局决胜还关系到确定性

只比 f 时，平局由堆的内部实现决定。同一份代码换一个 STL、换一种堆实现，走出的路径就可能不同（E9：40 次查询里 40 次不同）。对单机游戏这只影响表现，对帧同步游戏则是不同步 bug。第 7 章会给出完整的确定性方案。

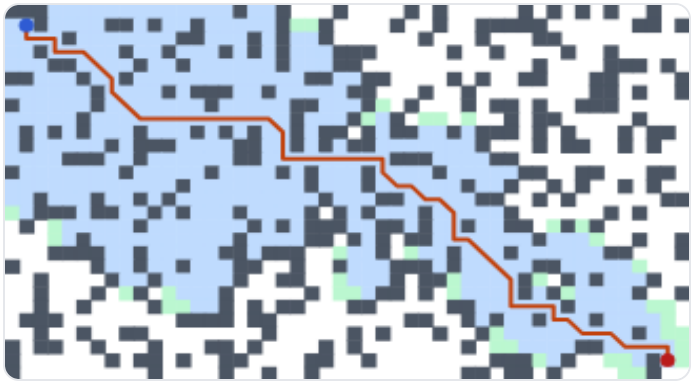
3.6 交互实验：启发函数和权重

六个面板分别是 Dijkstra、欧几里得、Octile、曼哈顿、加权 A*（w = 2）和贪心。注意标题里的“比最优多 x%”：只有前三个永远是最优的。

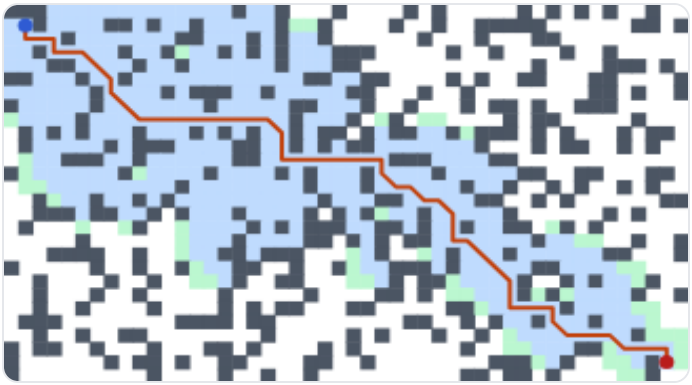
交互实验 • 同一张图，不同算法搜了多大范围



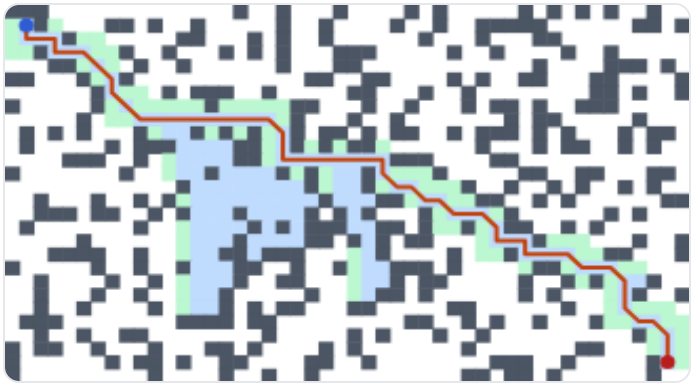
Dijkstra 展开 **961** 代价 62.38（最优）



A* • 欧几里得 展开 **498** 代价 62.38（最优）



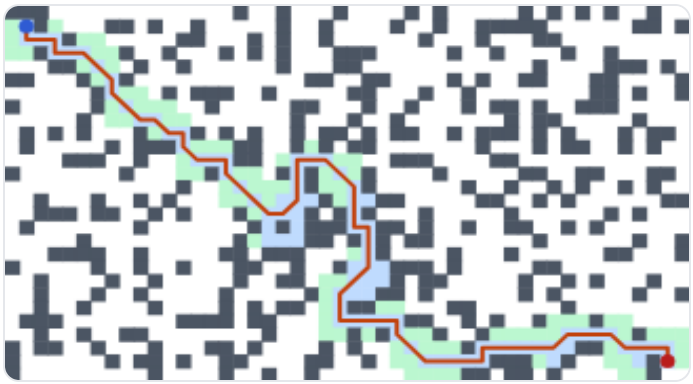
A* • Octile（八方向） 展开 **423** 代价 62.38（最优）



A* • 曼哈顿 展开 **142** 代价 62.38（最优）



加权 A*（w=2）• Octile（八方向） 展开 **98** 代价 65.46（比最优多 4.9%）



贪心最佳优先 • Octile（八方向） 展开 **87** 代价 75.46（比最优多 21.0%）

open（待展开） closed（已展开） 当前节点 JPS 扫描过 路径 障碍

本章速记

- **可采纳** h ≤ h* → 保证最短；**一致** h(n) ≤ c(n,n') + h(n') → 出队即最优、不用重开。一致 ⇒ 可采纳。
- 4 方向用曼哈顿，8 方向用 **Octile**，任意角度用欧几里得；8 方向下曼哈顿会高估（E2：最多长 9.8%）。
- 有地形代价时，h 要乘最便宜地形的单价；改了代价模型就要重新检查可采纳性。
- 加权 A*：代价 ≤ w × 最优；w = 1.2 就能少展开 90%（随机图），但对凹形陷阱无效。
- 平局决胜：f 相同取 h 小；开阔地上展开数差 47 倍（228 对 10772）。

▶ **自测：两个可采纳启发函数 h_1、h_2，$\max(h_1, h_2)$ 可采纳吗？更好吗？**

▶ **自测：4 方向地图用欧几里得距离，结果最短吗？效率如何？**

CHAPTER 04

为什么 A* 是对的，以及它最慢的时候

一段反证法说明最优性，一张图说明它为什么比 Dijkstra 搜得少，再看最坏情况：终点根本到不了。

本章目标

- 能在面试里用 30 秒讲清 A* 最优性的证明
- 理解“f < C* 的节点必展开、f > C* 的节点必不展开”，以及 A* 的最优效率
- 说出时间、空间复杂度，以及不可达时的应对手段

4.1 最优性证明（h 可采纳）

记最短路径代价为 C*。**反证**：假设 A* 取出终点 G 时 g(G) > C*。

1. 此时，最短路径上一定还有一个节点 n 在 open 里，并且 g(n) 已经等于它的最短距离 g*(n)。（起点在最短路径上；最短路径上第一个还没被展开的节点，它的前驱已被展开过，所以它一定已经以最优的 g 进入了 open。）
2. 由可采纳性：f(n) = g*(n) + h(n) ≤ g*(n) + h*(n) = C*。
3. 而 f(G) = g(G) + 0 > C* ≥ f(n)。
4. A* 每次取 f 最小的节点，n 的 f 更小，应该先于 G 被取出，矛盾。

所以 G 被取出时 g(G) = C*。**这也解释了为什么必须在出队时判断终点**：入队时 G 的 f 可能大于 C*，那时还有 f 更小的节点没展开。

面试速答版

“反证：如果取出终点时它的 g 比最优大，那最短路径上一定还有一个节点在 open 里。因为 h 不高估，这个节点的 f 不超过最优值，比终点的 f 小，按规则它应该先出队，矛盾。”

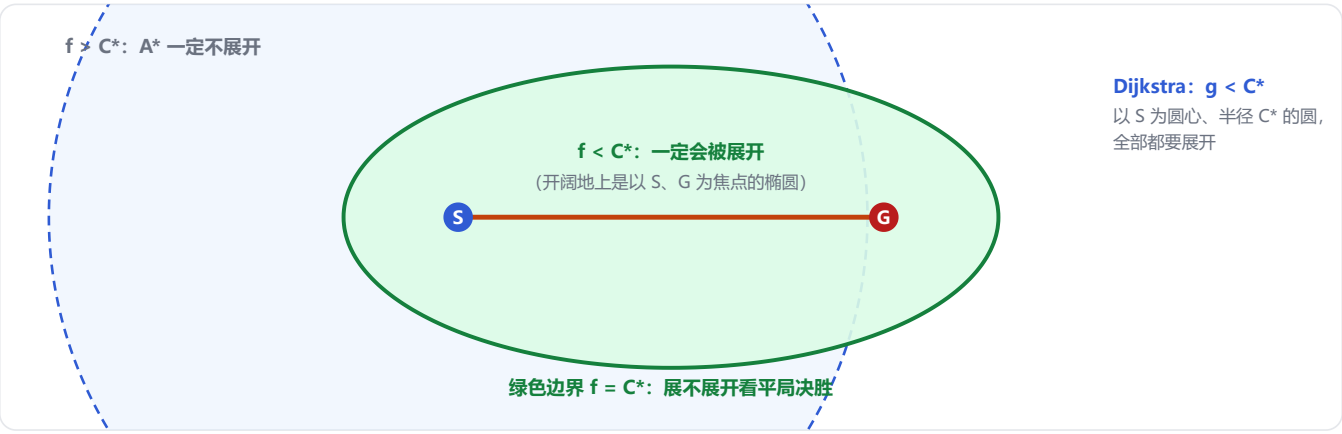
4.2 一致性带来的两个好处

若 h 一致，对任意边 (n, n')：f(n') = g(n) + c(n, n') + h(n') ≥ g(n) + h(n) = f(n)。也就是说**沿着任何一条边走，f 都不会变小**。由此：

- **被展开节点的 f 序列单调不减**（第 2 章手算时看到的 40, 40, 48, 48, 54…）。
- **节点第一次出队时 g 就是最优的**，以后不可能再找到更短的路。所以 closed 节点不必重开，惰性删除版只需“出队时跳过已 closed 的条目”。

只可采纳、不一致的 h 下，一个节点可能先以偏大的 g 出队，之后又被更短的路径更新。这时要么允许**重开**（从 closed 移回 open，最坏会导致指数次展开），要么接受次优。实际项目里用的几何距离都是一致的，这个问题一般只出现在自己设计的“组合启发函数”里。

4.3 A* 到底省在哪：f = C* 这条等值线



h 越接近 h*，绿色区域越扁、越贴近 S—G 这条线；h = 0 时它就膨胀成蓝色的圆

图 4-1　A* 的展开范围：f < C* 的节点必展开，f > C* 的必不展开；这就是它比 Dijkstra 省的部分

由 4.1 的论证还能推出：**任何使用同一个可采纳启发函数、并保证找到最短路的算法，都必须展开所有 f < C* 的节点**，否则它无法排除“那里藏着一条更短的路”。A* 恰好只展开这些节点（加上一部分 f = C* 的节点），所以说 A* 是**最优效率**（optimally efficient）的。想比 A* 更快，只有两条路：**换一个更准的 h**，或者**放弃最优**（加权、分层、JPS 之类的剪枝不算放弃最优，它们改变的是图本身）。

4.4 复杂度

维度	结论	说明
时间（图上）	**O(E log V)**	每个节点最多展开一次（一致 h），每条边最多引起一次入堆；二叉堆每次操作 log。格子图 E ≈ 8V，所以是 O(V log V)
时间（树搜索视角）	最坏 $O(b^d)$，b 为分支数，d 为解的深度	h 完美时降到 O(d)；h 误差有界（$\|h - h^*\| \le O(\log h^*)$）时是多项式级
空间	**O(V)**	open + closed 要存下所有展开过的节点。**A* 的瓶颈通常是内存，而不是时间**
最好情况	O(d)	开阔地 + 完美启发 + 正确的平局决胜（E1：开阔地 266 格长的路只展开 228 个节点）

深入一层：省内存的变体

IDA*（迭代加深 A*）：以 f 为阈值做深度优先搜索，超过阈值就回溯，每轮把阈值提高到上一轮被剪掉的最小 f。内存只有 O(d)，但会大量重复展开，在有环的格子图上很慢，适合 15 数码这类状态空间巨大的谜题。**SMA***：内存满时丢掉 f 最大的叶子。游戏寻路里几乎不用它们，更常见的做法是把图变小（导航网格、分层）。

4.5 最坏情况：终点不可达

A* 只有在 open 被取空时才能断定“无路可走”，而这意味着**起点所在的整个连通块都被展开了一遍。**启发函数在这里完全帮不上忙，还会因为堆变大而比 BFS 更慢。

E8 · 随机障碍 25%，终点在一个被围死的小区域里

```
A* 直接搜：展开 195130 个节点（= 起点所在连通块 195130 格），耗时 20.529 ms，结果：无路径
连通块预计算：加载时 flood fill 一次 3.602 ms（946 个连通块）；之后每次判定 0.0003 us（一次数组比较）
展开上限 20000：2.681 ms 后放弃，可退而求其次走向“h 最小的已探索点”
```

20 ms 已经超过一帧（60 FPS 下 16.7 ms）。点击一次“走到那个被围起来的位置”，游戏就卡一下。四种应对手段：

① 连通块编号

② 展开上限

加载地图时 flood fill，给每个格子标上所属区域编号。查询前先比较编号，不同就直接返回不可达，O(1)。动态障碍改变连通性时要局部重算，导航网格可以在多边形上维护。

超过 N 个节点就放弃。比较安全，但可能把“可达但很远”误判为不可达，所以要和 ① 搭配使用。

③ 退而求其次

搜索失败时，返回已展开节点中 h 最小（离终点最近）的那个，让单位走到“能走到的最近处”。玩家点了一个不可达的位置时，这通常就是期望的表现。

④ 终点吸附

玩家点的位置在障碍里时，先把终点吸附到最近的可走点，再寻路。Detour 的 `findNearestPoly` 就做这件事。

面试注意

“A* 最坏情况是什么”的标准答案不是 $O(b^d)$ 这种教科书公式，而是**“终点不可达，要把整个连通块搜完；还有启发函数被凹形障碍误导，要把陷阱填满”**，然后给出对应手段。这体现的是工程判断。

本章速记

- **最优性**：反证，最短路径上总有一个 f ≤ C* 的节点在 open 里，终点不可能以更大的 g 先出队。
- **一致性** ⇒ f 沿边不减 ⇒ 出队即最优 ⇒ closed 不用重开。
- A* 展开所有 f < C* 的节点，不展开 f > C* 的节点，是**最优效率**的。
- 复杂度：时间 O(E log V)，空间 O(V)，内存往往是瓶颈。
- 最坏情况是**不可达**（E8：20.5 ms 搜完 19.5 万格）→ 连通块编号 O(1) 预判 + 展开上限 + 返回最近点。

▶ **自测：证明里为什么要求“最短路径上的那个节点 g 已经是最优的”？**

▶ **自测：双向 A* 能更快地发现“不可达”吗？**

PART THREE

第三部分　工程实现与加速算法

教科书上的 A* 能跑，但离“每帧几十次查询、不卡顿”还有距离。先把数据结构做对，再从图本身下手：JPS 剪掉对称路径，HPA* 把图变小，平滑让路径变自然。

CHAPTER 05

open list 与节点状态：数据结构决定常数

同一个算法、同样的展开次数，数据结构选错可以慢 2.8 倍，复位方式选错可以慢 90 倍。

本章目标

- 比较 open list 的四种实现，理解 decrease-key 与惰性删除的取舍
- 理解“每次查询清零整图数组”为什么是短查询的性能杀手，掌握 generation 计数
- 写出可以直接用在项目里的工程版 GridPathfinder

5.1 A* 的时间花在哪

每展开一个节点要做：一次出堆（log n）、8 次邻居可走性检查、若干次 g 比较和入堆。另外每次查询开始前，要让所有节点的状态回到“未访问”。前者决定长查询的耗时，后者决定短查询的耗时，而**游戏里绝大多数查询都是短查询**：追击目标、走到旁边的掩体、技能落点附近的绕行。

5.2 open list 的四种实现

实现	取最小	插入	改 key（发现更短的 g）	特点
二叉堆 + 惰性删除	O(log n)	O(log n)	直接再插一份，出队时跳过过期条目	最简单，`std::priority_queue` 即可
索引二叉堆	O(log n)	O(log n)	O(log n) 原地上浮	要维护“节点 → 堆中位置”数组；堆里没有重复
`std::set`	O(log n)	O(log n)	删除旧键再插入	红黑树，每个节点单独分配内存，缓存不友好
无序数组	O(n) 扫描	O(1)	O(1) 原地改	open 小时反而快；open 大时退化为 $O(n^2)$

E5：随机障碍 25%，512×512	长查询（平均展开 11757）	短查询（≤16 格，平均展开 49）
priority_queue + 惰性删除	**1.301 ms**	5.60 us
索引二叉堆 + decrease-key	1.388 ms	5.54 us
std::set + 删除再插入	**2.212 ms**	**9.22 us**
无序数组 + 线性找最小	**3.593 ms**	**5.01 us**

- **惰性删除和索引堆几乎一样快。**教科书强调 decrease-key，但在格子图上同一节点被改 key 的次数很少，堆里多出的过期条目代价不大；而索引堆每次交换都要多写一次位置数组。所以**工程上的默认选择就是 priority_queue + 惰性删除。**
- **std::set 慢 70%**：红黑树每个节点单独分配内存，指针跳转多。
- **无序数组在短查询上最快**，因为 open 里只有几十个元素，线性扫描完全在 L1 缓存里，没有堆调整的开销；长查询时 open 有上千个元素，就慢了 2.8 倍。这再次说明：**先看你的查询分布，再选数据结构。**

深入一层：整数代价可以用桶队列

代价是小整数时，可以把 f 值相同的节点放进同一个桶（数组下标就是 f），取最小只要从当前 f 往后找第一个非空桶，插入和取出都是 O(1)。一致 h 下 f 单调不减，所以“当前桶”的指针只会前进，这叫 **bucket queue / Dial 算法**；值域大时可以用基数堆（radix heap）。它们在大规模寻路服务（比如服务器批量寻路）里能比二叉堆快不少，代价是要预估 f 的范围。另一个常见的小优化是 **4 叉堆**：树更矮，一个节点的孩子在内存里相邻。

5.3 节点状态怎么复位

每次查询都需要一张“这个节点的 g / parent / 是否 closed”的表。三种做法：

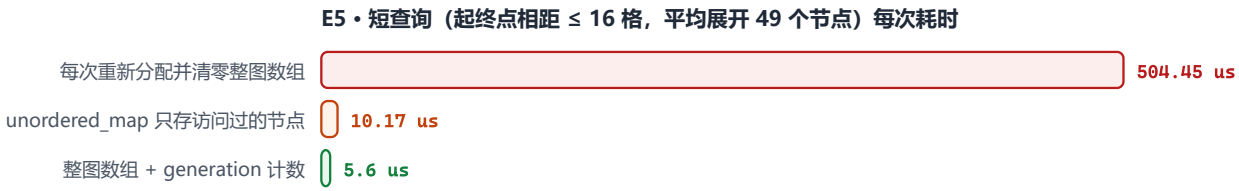


图 5-1 512×512 的图，每次查询清零 g / parent / 状态三个数组约 2.4 MB；只展开 49 个节点的查询，90% 以上的时间都花在清零上

做法	长查询	短查询	说明
整图数组 + generation 计数	**1.297 ms**	**5.60 us**	数组只分配一次。每个节点另存一个 `gen` 戳，`gen[i] == 当前查询号` 才算有效；开始新查询只要把查询号加 1，O(1) 复位
每次重新分配并清零	1.763 ms	**504.45 us**	面试版代码就是这样写的。复位代价 O(V)，和查询长短无关
`unordered_map`	**2.354 ms**	10.17 us	只为访问过的节点分配，复位便宜；但每次访问都要哈希、可能分配内存

面试注意：这是区分“写过 demo”和“做过项目”的点

被问“你的 A* 怎么优化”时，除了 JPS、分层这些算法层面的回答，能说出“节点状态用 generation 计数复位，避免每次查询清零整张图；open 用惰性删除的二叉堆，容器保留容量不重新分配”，会显得很有工程经验。还要注意 gen 是 32 位计数，**回绕到 0 时要整体清零一次**，否则旧数据会被误认为有效。

5.4 其他常数优化

- **邻居偏移预计算**：8 个方向的下标偏移 `{1, -1, W, -W, W+1, …}` 提前算好，不要每次做乘法。
- **地图加一圈边框**：在地图外围加一圈“障碍”，邻居检查就不用判断越界。
- **压缩节点状态**：g 用 int32 就够（512×512 迷宫最长路径 36968 格 × 1414 ≈ 5200 万）；可走性用位图；parent 可以只存 3 比特的方向而不是 int 下标。内存越小，缓存命中率越高。
- **SoA 布局**：g、parent、gen 分别放在连续数组里，热循环只碰需要的字段。
- **容器复用**：open 的 vector 每次 `clear()` 而不是重新构造，保留容量。

5.5 工程版 GridPathfinder

把上面的要点组合起来：generation 复位、惰性删除二叉堆、全序比较器（为帧同步准备）、禁止切角、展开上限。代码在 `demos/astar_engineering.cpp`，在 256×256 随机图上和 Dijkstra 对比了 300 次查询，代价全部一致（`engineering_output.txt`：294 次最短且正确，6 次不可达，0 错误，平均 0.446 ms）。

GridPathfinder（工程版，可直接用于项目；demo 中另有 cost() / lastExpanded() 两个访问器，此处省略）

```
class GridPathfinder {
public:
    explicit GridPathfinder(const Grid& grid)
        : grid_(grid), n_(grid.w * grid.h), g_(n_), parent_(n_), seenGen_(n_, 0), closedGen_(r
        const int W = grid.w;
        const int off[8] = {1, -1, W, -W, W + 1, -W + 1, W - 1, -W - 1};    // 与 DX/DY 顺序一致
        std::copy(off, off + 8, offset_);
        heap_.reserve(4096);
    }

    // 找到返回 true，path 为格子下标序列（含起终点）；maxExpand 限制单次查询的展开数
    bool find(int s, int t, std::vector<int>& path, int maxExpand = INT_MAX) {
        if (++gen_ == 0) {                                      // 40 亿次查询后回绕：整体清零一次
            std::fill(seenGen_.begin(), seenGen_.end(), 0u);
            std::fill(closedGen_.begin(), closedGen_.end(), 0u);
            gen_ = 1;
        }
        heap_.clear();                                          // 保留容量，不重新分配
        path.clear();
        const int W = grid_.w, tx = t % W, ty = t / W;
        auto h = [&](int i) {                                   // Octile，直走 1000、斜走 1414
            int dx = std::abs(i % W - tx), dy = std::abs(i / W - ty);
            return 1000 * std::max(dx, dy) + 414 * std::min(dx, dy);
        };
        push(s, 0, -1, h(s));
        int expanded = 0;
        while (!heap_.empty()) {
            std::pop_heap(heap_.begin(), heap_.end(), Cmp{});
            OpenE e = heap_.back(); heap_.pop_back();
            if (closedGen_[e.i] == gen_) continue;              // 惰性删除
            closedGen_[e.i] = gen_;
            if (e.i == t) { build(t, path); return true; }
            if (++expanded > maxExpand) break;
            const int x = e.i % W, y = e.i / W;
            for (int d = 0; d < 8; ++d) {
                if (!canMove(x, y, d)) continue;
                const int j = e.i + offset_[d];
                if (closedGen_[j] == gen_) continue;
                const int ng = g_[e.i] + (d < 4 ? 1000 : 1414);    // int32 足够：最长路径 × 141
                if (seenGen_[j] == gen_ && ng ≥ g_[j]) continue;
                push(j, ng, e.i, h(j));
            }
        }
```

```
        return false;
    }

private:
    struct OpenE { int f, h, i; };
    struct Cmp {   // 全序：f 小 → h 小 → 下标小。任何平台、任何堆实现结果都相同
        bool operator()(const OpenE& a, const OpenE& b) const {
            if (a.f != b.f) return a.f > b.f;
            if (a.h != b.h) return a.h > b.h;
            return a.i > b.i;
        }
    };
    static constexpr int DX[8] = {1, -1, 0, 0, 1, 1, -1, -1};
    static constexpr int DY[8] = {0, 0, 1, -1, 1, -1, 1, -1};
    bool canMove(int x, int y, int d) const {
        if (!grid_.walkable(x + DX[d], y + DY[d])) return false;
        return d < 4 || (grid_.walkable(x + DX[d], y) && grid_.walkable(x, y + DY[d]));   // 斜
    }
    void push(int i, int g, int p, int h) {
        seenGen_[i] = gen_; g_[i] = g; parent_[i] = p;
        heap_.push_back({g + h, h, i});
        std::push_heap(heap_.begin(), heap_.end(), Cmp{});
    }
    void build(int t, std::vector<int>& path) const {
        for (int i = t; i != -1; i = parent_[i]) path.push_back(i);
        std::reverse(path.begin(), path.end());
    }

    const Grid& grid_;
    int n_;
    std::vector<int> g_, parent_;
    std::vector<uint32_t> seenGen_, closedGen_;   // == gen_ 才算本次查询的有效数据
    uint32_t gen_ = 0;
    std::vector<OpenE> heap_;
    int offset_[8];
};
```

5.6 多线程寻路

寻路是只读地图、彼此独立的计算，很适合放到工作线程：

- **每个线程一份搜索状态**：GridPathfinder 里的 g / parent / gen / heap 都是可变状态，不能共享，每个工作线程各持有一个实例。地图数据只读共享。Detour 的 `dtNavMeshQuery` 也是这个设计：navmesh 共享，query 对象每线程一个。
- **请求队列 + 回调**：游戏逻辑提交请求，下一帧或几帧后拿到结果。UE 的 `FindPathAsync` 就是这种异步接口。
- **地图修改要同步**：动态障碍改变地图时，要么加读写锁，要么用双缓冲（逻辑帧末尾交换），保证搜索过程中地图不变。
- **帧同步游戏要小心**：异步寻路的结果必须在一个**确定的逻辑帧**生效（比如“请求帧 + 2”），而不是“哪一帧算完就哪一帧用”，否则各端时序不同就会不同步。

本章速记

- open：**priority_queue + 惰性删除**是默认选择，和索引堆一样快（1.30 对 1.39 ms），std::set 慢 70%；open 很小时线性数组最快。
- 节点状态：**整图数组 + generation 计数**，O(1) 复位；每次清零整图会让短查询慢 90 倍（504 对 5.6 us）。gen 回绕时整体清零一次。
- 常数优化：邻居偏移预计算、外围加边框、int32 g、SoA、容器复用。
- 多线程：地图只读共享、搜索状态每线程一份；帧同步下异步结果必须在确定的逻辑帧生效。

▶ **自测：惰性删除会让堆变多大?**

CHAPTER 06

加速与平滑：JPS、分层寻路、双向搜索、拉绳

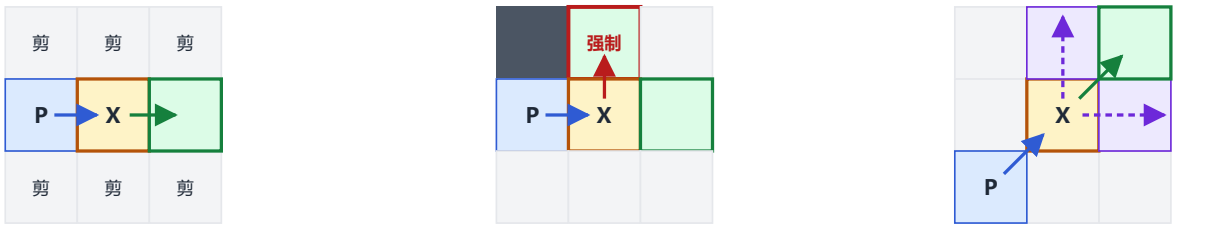
A* 已经是最优效率的，想更快只能改变图本身：剪掉对称的路径，把图变小，或者预先算好一部分。

本章目标

- 理解 JPS 的“对称路径剪枝”思想、跳跃规则和适用条件
- 说清 HPA* 的预处理、抽象图搜索、细化三步，以及它的取舍
- 知道双向 A*、ALT 的思路和坑
- 会用拉绳（视线检测）把锯齿形的格子路径变平滑

6.1 JPS：跳过对称路径

开阔的格子地图上，两点之间有大量代价相同的路径，比如“先右后下”和“先下后右”。A* 会把这些等价路径上的节点都放进 open。**JPS（Jump Point Search，Harabor & Grastien，2011）**的观察是：沿着一个方向走时，大多数邻居经过当前格到达并不比绕别的路更好，可以直接剪掉。于是它沿直线或对角线一路“跳”过去，只在必须拐弯的地方（跳点）停下来、放进 open。



① 直线走：只保留正前方
其余邻居从 P 出发不经过 X 也能同样好地到达

② 强制邻居：X 是跳点
P 上方是墙，P 无法斜走到 X 上方（禁止切角），只能经过 X → 必须停下

③ 斜着走：先做两次直线扫描
向右、向上各扫一条直线，任一条找到跳点，X 就是跳点；否则再斜走一步

停下的条件只有三种：到达终点、出现强制邻居、斜向扫描时直线方向发现了跳点；其余格子不入堆

图 6-1 JPS 的三条规则（本书实现的是“禁止切角”的版本，和第 1 章的移动规则一致）

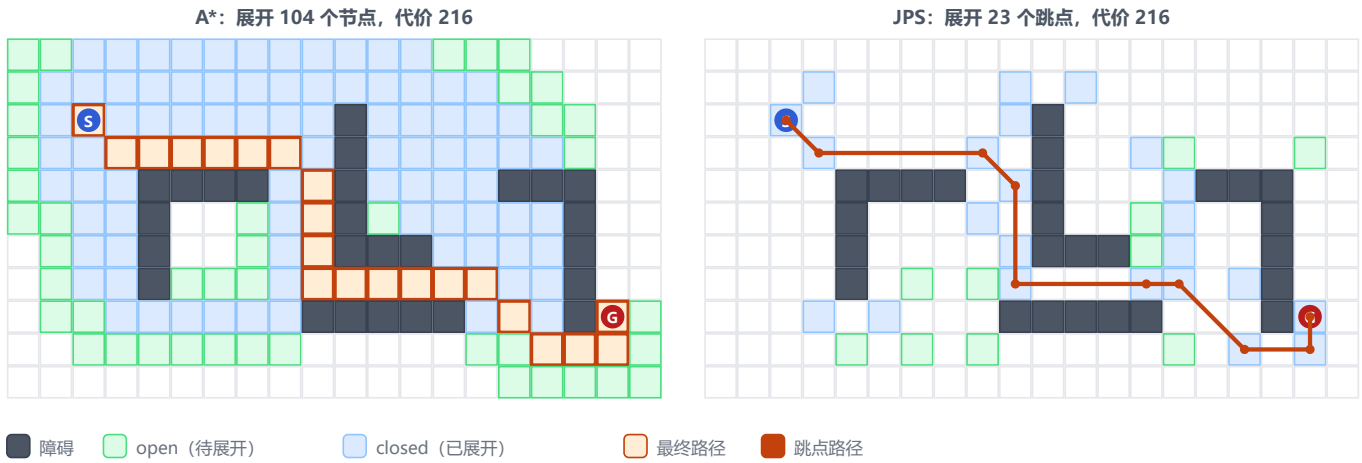


图 6-2　同一张图：A* 展开 104 个节点，JPS 只展开 23 个跳点，路径代价相同（216）。JPS 面板里蓝色格子是被展开的跳点，橙色折线连接的是路径上的跳点

伪代码

JPS · 伪代码

```
JPS(S, G):  与 A* 相同的主循环，只是“邻居”换成“跳点”
    cur = open 中 f 最小的节点
    dirs = prune(cur, 从 parent 过来的方向)        // 图 6-1 的剪枝规则；起点取全部 8 个方向
    for d in dirs:
        jp = jump(cur, d)
        if jp 存在：按 A* 规则更新 g[jp] = g[cur] + octile(cur, jp)，放入 open

jump(x, d):                                          // 沿方向 d 一直走
    loop:
        x = x + d
        if x 是障碍：return 无
        if x == G：return x
        if d 是直线 且 x 有强制邻居：return x
        if d 是斜向 且 (jump(x, d 的横向分量) 或 jump(x, d 的纵向分量) 有结果)：return x
        if d 是斜向 且 两侧直邻格不全可走：return 无      // 禁止切角
```

实测 E6：JPS 什么时候快

地图	A* 展开	A* 耗时	JPS 跳点	JPS 扫描格子	JPS 耗时	加速比
开阔地	228	0.029 ms	3	**263620**	0.444 ms	**0.1×（更慢）**
随机障碍 25%	11758	1.385 ms	6285	49138	0.919 ms	1.5×
迷宫	79476	3.476 ms	23927	85316	1.325 ms	2.6×
房间	7822	0.953 ms	564	64090	0.214 ms	**4.5×**
U 型陷阱（E1）	52359	3.934 ms	7	—	0.524 ms	**7.5×**

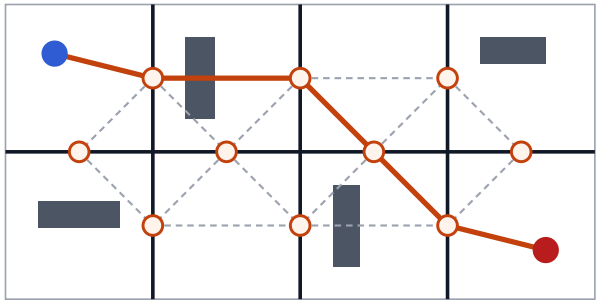
- **JPS 不是总能加速。**开阔地上 A* 配合正确的平局决胜只展开 228 个节点，几乎是完美的；而 JPS 每次斜向一步都要向两个方向做直线扫描，扫描线会一直延伸到地图边缘，最后扫了 26 万格，反而慢了 15 倍。
- **JPS 最擅长的是“大片开阔区域 + 若干障碍”**：房间、U 型陷阱这类地图，A* 要把陷阱里的对称路径全部入堆，JPS 只扫描不入堆，堆操作少了一两个数量级。
- **JPS 保证最短**（E6 中 100% 最优），因为它剪掉的只是对称的等价路径。

面试注意：JPS 的适用条件

- **只适用于均匀代价的格子图**：剪枝依赖“所有可走格代价相同”。有地形权重（沼泽、道路）就不能直接用。
- **不适用于导航网格、路点图**：那些图没有“直线方向”的概念。
- **JPS+**（Steve Rabin）预先计算每个格子在 8 个方向上到最近跳点或墙的距离，把扫描变成查表，通常比 JPS 快一个数量级以上；代价是地图一变就要重算受影响的行列，适合静态地图。

6.2 HPA*：把大图变小

A* 的代价和图的规模直接相关。**HPA*（Hierarchical Pathfinding A*，Botea 等，2004）**把地图切成若干簇（cluster），只在簇的边界上选出“入口”节点，预先算好同一个簇内入口之间的距离，形成一张小得多的抽象图：



粗黑线：簇边界　橙点：入口节点　虚线：簇内预计算的入口间距离
橙色折线：在抽象图上 A* 得到的入口序列

① 预处理（加载地图时，一次）
切簇 → 在相邻簇的公共边上选入口 → 每个簇内用 A* / Dijkstra 算出入口两两之间的距离，作为抽象图的边

② 查询：插入起终点
把 S、G 临时连到所在簇的入口（各做一次簇内搜索）

③ 抽象图上做 A*
节点只有几十上百个，极快

④ 细化（可以按需、分段做）
相邻两个入口之间在簇内做一次小 A*，拼出完整路径；
单位只需要前两段就能出发，后面边走边算

图 6-3　HPA* 的四个步骤。可以叠加多层：簇再组成更大的簇

- **收益**：大地图上查询快一到两个数量级；内存只多了抽象图；细化可以分段做，降低首帧延迟。
- **代价**：结果通常比最短路长几个百分点（入口位置是固定选的），所以一般最后还要做平滑；地图修改时要重算受影响的簇。
- 导航网格本身也是一种“分层”：多边形已经比格子粗很多。超大世界还会在 navmesh tile 之上再建一层图（比如城镇、道路网络）。

6.3 双向 A* 与预计算启发

双向 A*

从起点和终点同时搜索，两边相遇时停止。直觉上两个半径减半的圆面积之和只有一个大圆的一半。

坑：两边第一次相遇时得到的路径**不一定最短**。正确的停止条件是：已找到的最好路径长度 μ ≤ 两侧 open 中最小 f 的较大者（或使用更细的双向搜索终止条件）。实现复杂，在游戏里用得不多；它更大的价值是能更快地发现“终点被困在小区域里”。

ALT（地标 + 三角不等式）

预先选 k 个地标 L，用 Dijkstra 算出每个地标到所有节点的距离 d(L, ·)。查询时：

$$h(n) = \max_L |d(L, G) - d(L, n)|$$

由三角不等式，它可采纳且一致，通常比几何距离准得多，迷宫、绕路多的地图上效果明显。代价是内存 k × V。

完全预计算

节点少（如路点图几百个点）时，直接用 Floyd 或 n 次 Dijkstra 算出全源最短路的“下一跳表”，查

询时 O(路径长度) 查表即可。内存 O(V²)，只适合小图。

6.4 路径平滑：拉绳

格子图上 8 方向的最短路，在真实空间里并不是最短的：它只能走 0°、45°、90° 这几个角度，看起来像锯齿。**拉绳（string pulling）**的思路很简单：从起点出发，沿着原始路径往后找，只要还能“看见”（中间没有障碍），就跳过中间的路点；看不见了，就把上一个看得见的点作为新的拐点。

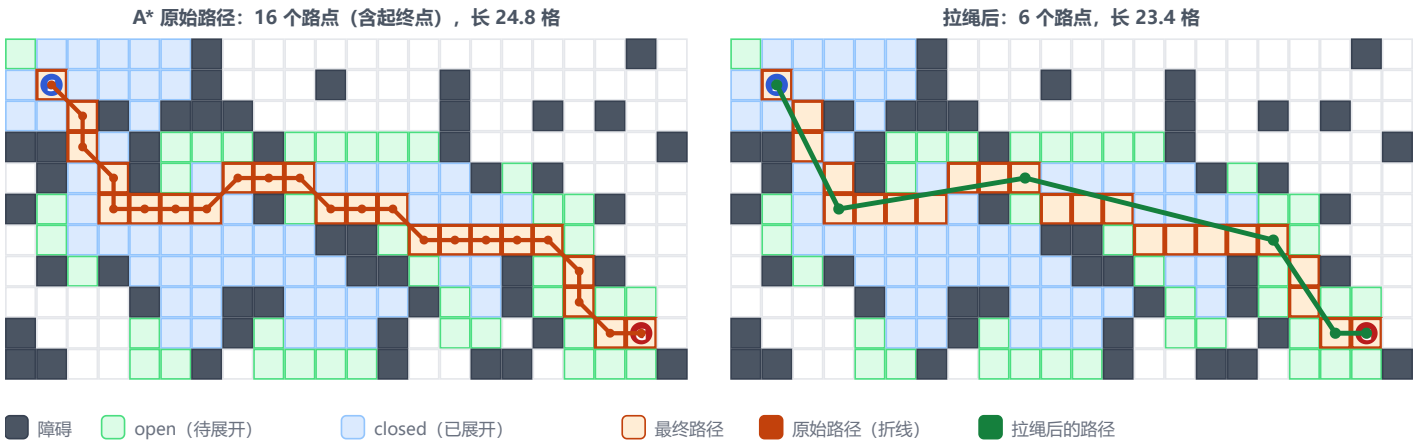


图 6-4 拉绳：绿色折线只在必须拐弯的地方拐弯。视线检测和寻路使用同样的“禁止切角”规则，平滑后的路径不会擦过墙角

拉绳：贪心找最远可见点

```
// los：格子级视线检测，线段经过格角时，两侧格子都必须可走（与禁止切角一致）
std::vector<int> string_pull(const Map& m, const std::vector<int>& path) {
    std::vector<int> out = {path[0]};
    size_t a = 0;
    while (a + 1 < path.size()) {
        size_t b = a + 1;
        while (b + 1 < path.size() && los(m, path[a], path[b + 1])) ++b;   // 能看见就继续往后
        out.push_back(path[b]);
        a = b;
    }
    return out;
}
```

E10：40 条路径平均	原始路点	拉绳后路点	原始长度	拉绳后长度	缩短	平滑耗时
开阔地	3.0	2.0	266.6	252.3	5.36%	0.033 ms
随机障碍 25%	103.6	63.5	282.9	272.3	3.73%	0.005 ms
房间	26.1	15.0	280.0	269.7	3.66%	0.007 ms

平滑的耗时不到寻路的 1%（随机图上 A* 1.3 ms，平滑 0.005 ms），路径短了 4%～5%，拐点少了 40%。开阔地的平滑反而最慢，是因为直线很长，每次视线检测要走过很多格子。

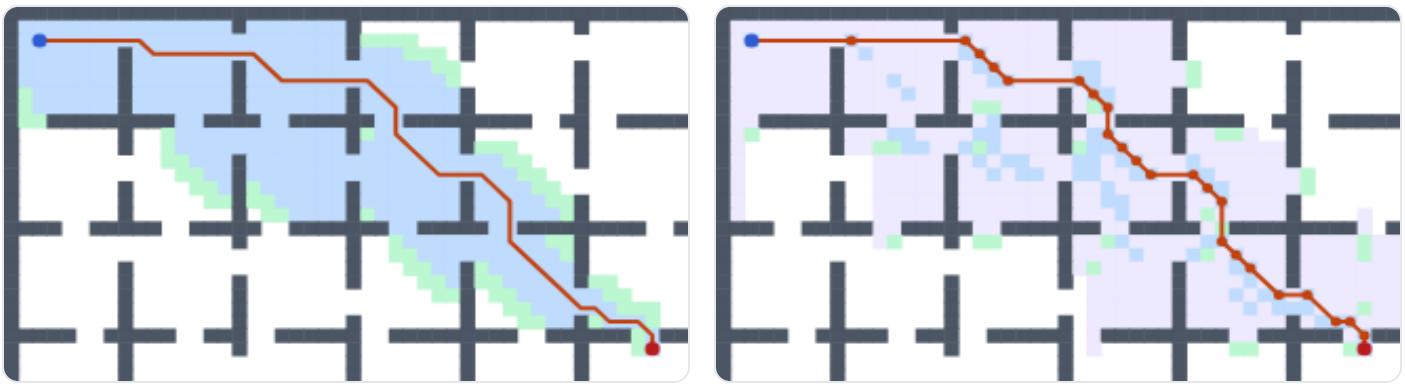
深入一层：Theta* 与“任意角度”寻路

拉绳是寻路之后的后处理，结果仍然受原始路径走向的约束，不一定是真实空间里的最短路。**Theta***（Nash 等，2007）把视线检测放进搜索过程：更新邻居时，如果邻居能直接看见当前节点的 parent，就把邻居的 parent 设为“祖父”，g 用直线距离计算。这样搜出来的路径天然是任意角度的，代价是每次松弛都多一次视线检测。导航网格上对应的做法是第 7 章的漏斗算法。

6.5 交互实验：A* 对比 JPS

在“房间”和“U 型陷阱”上，JPS 只展开少数几个跳点（蓝色），浅紫色是它沿直线扫描过但没有入堆的格子；换到“空地”，扫描线铺满了半张图。

交互实验 • 同一张图，不同算法搜了多大范围

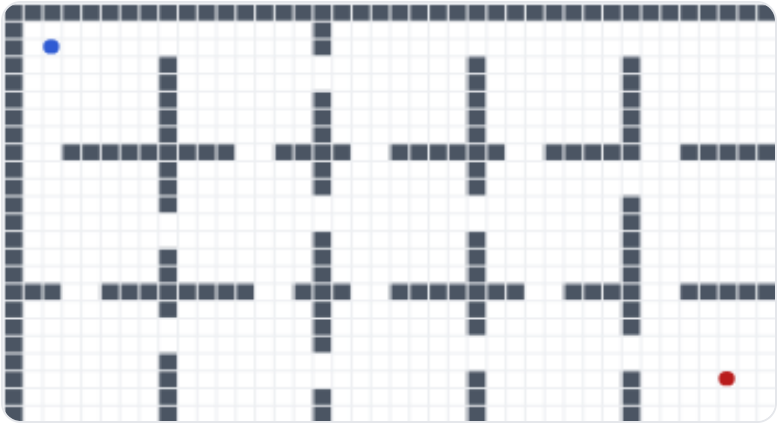


A* • Octile（八方向） 展开 **372** 代价 56.04（最优）　　**JPS 跳点搜索** 展开 **79** 代价 56.04（最优）

open（待展开）　closed（已展开）　当前节点　JPS 扫描过　路径　障碍

下面是 JPS 的单步演示：每展开一个跳点，消息栏会显示剪枝后剩几个方向、每个方向扫描了多少格、是否找到新的跳点。可以和 A* 切换着看。

交互实验 • JPS 单步演示（浅紫色 = 扫描过但没有入堆的格子）



```
open = {S}
while open 非空:
  cur = open 中 f 最小的跳点
  if cur == G: 依次连接各跳点得到路径
  dirs = 根据“从哪个方向来”剪枝后的方向
  for d in dirs:
    jp = jump(cur, d)
    // 沿 d 直走，遇到强制邻居或终点才停
    if jp 存在且不在 closed: 更新 g、open
return 无路径
```

点击「播放」或「单步」开始；鼠标悬停在格子上可查看 g / h / f。

open（待展开） closed（已展开） 当前节点 JPS 扫描过 路径 障碍

6.6 加速手段汇总

手段	改变了什么	保证最短	需要预处理	动态障碍	适用
更好的 h / 平局决胜	展开顺序	✓	—	✓	所有
加权 A*	展开顺序	**≤ w 倍**	—	✓	所有
JPS	剪掉对称路径	✓	—	✓	均匀代价格子图
JPS+	扫描查表	✓	✓	**局部重算**	静态格子图
HPA*	图变小	**近似**	✓	**重算簇**	大地图
导航网格	图变小	**近似最短**	✓（离线烘焙）	**tile 重建**	3D 游戏
ALT	更准的 h	✓	✓	✗	绕路多的静态图
拉绳 / Theta*	路径形状	更接近真实最短	—	✓	格子图

本章速记

- **JPS**：剪掉对称路径，只把跳点入堆；保证最短；只适用均匀代价格子图。房间图快 4.5 倍，开阔地反而慢（扫描铺满全图）。JPS+ 预计算跳跃距离。
- **HPA***：切簇 → 入口 → 抽象图 A* → 分段细化；快一到两个数量级，结果近似最短。
- **双向 A***：第一次相遇不一定最短；**ALT**：地标 + 三角不等式，h 更准，内存 k×V。
- **拉绳**：贪心跳到最远可见点，路点少 40%、长度短 4%，耗时不到寻路的 1%；视线检测要和切角规则一致。

PART FOUR

第四部分　游戏实战：导航网格、分帧、多单位与确定性

寻路在游戏里从来不是孤立的一次 A* 调用。它是一条流水线：离线烘焙导航数据，运行时排队、分帧、搜索、拉直、跟随、避让；帧同步游戏还要保证每一端算出的结果逐位相同。

CHAPTER 07

寻路在游戏里的样子

导航网格与漏斗算法、分帧寻路、动态障碍、流场、全局与局部的分层，以及帧同步下的确定性。

本章目标

- 说清 Recast/Detour 的烘焙流程和一次查询的三个步骤，理解漏斗算法
- 知道怎么把寻路分帧、怎么处理动态障碍、什么时候该用流场
- 分清全局寻路与局部避障的职责
- 能列出帧同步游戏里寻路的确定性检查清单

7.1 导航网格：3D 游戏的主流

UE 和 Unity 的内置寻路都是导航网格。UE 直接集成了开源的 **Recast/Detour**（作者 Mikko Mononen）：Recast 负责从场景几何体离线生成导航网格，Detour 负责运行时查询。

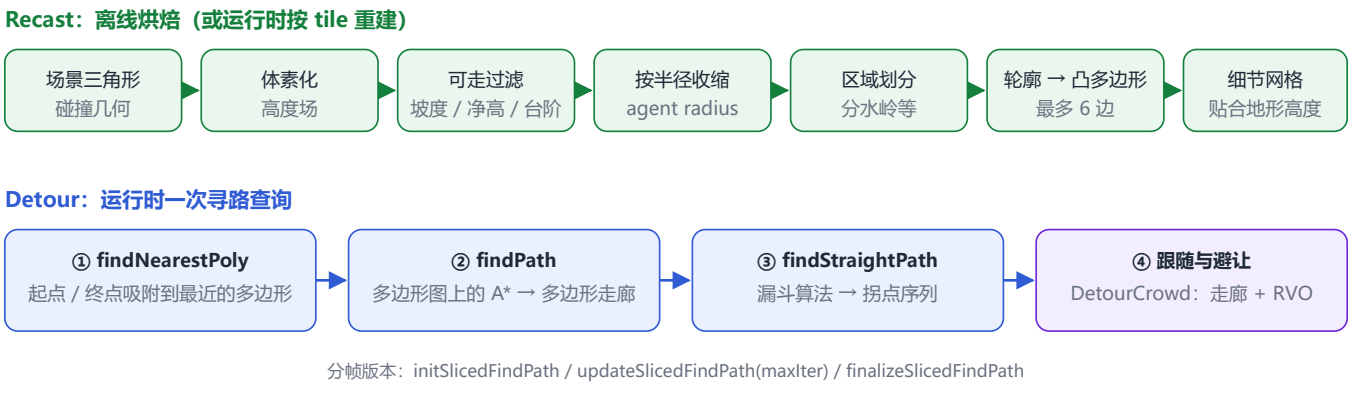


图 7-1　Recast/Detour 的两段式流程：烘焙一次，查询很多次

- **多边形图上的 A***：节点是凸多边形，邻居是共享边的多边形。Detour 的代价按“多边形边上的点之间的距离”累计，启发函数是到终点的欧几里得距离乘以 0.999（`H_SCALE`，留一点余量，防止浮点误差导致高估）。

- **结果是“近似最短”**：多边形内部可以自由走，但搜索时只用边上的固定点估算代价，所以搜出的多边形走廊不一定是真实空间里最短的那条，不过一般足够接近。
- **agent radius**：烘焙时把可走区域向内收缩一个角色半径，运行时就可以把角色当成一个点来寻路。不同体型的角色需要不同的 navmesh（UE 中叫 Supported Agents）。

7.2 漏斗算法：把多边形走廊拉成直线

A* 在 navmesh 上给出的是一串多边形，相邻多边形之间的公共边叫**入口**（portal）。最短路径一定是一条只在入口端点处拐弯的折线。**漏斗算法**（Simple Stupid Funnel Algorithm）用一次线性扫描求出它：

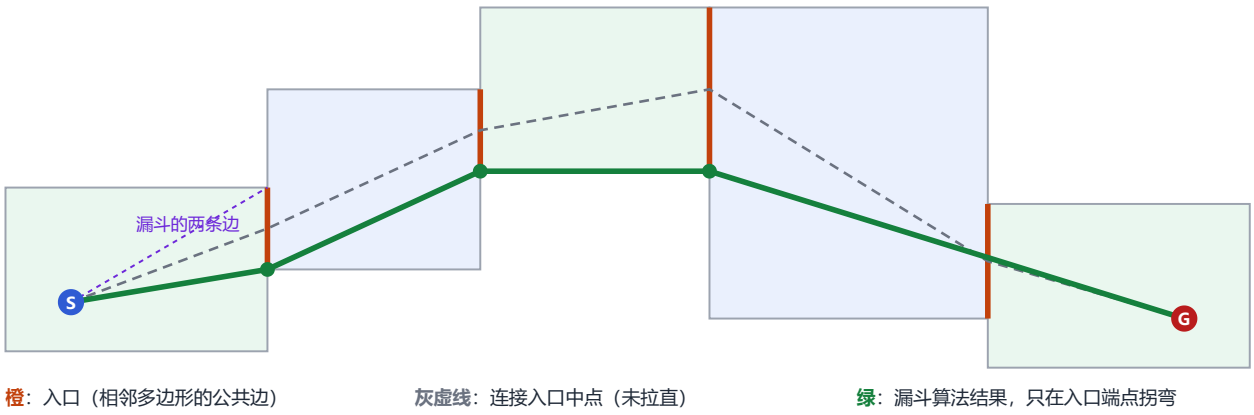


图 7-2 漏斗算法：从起点出发维护一个“漏斗”，随着入口一个个收窄，越不过去时漏斗的一侧端点就成为新的拐点

漏斗算法 · 伪代码（Simple Stupid Funnel）

```
apex = S；left = right = S                          // 漏斗顶点与左右两条边的端点
for 每个入口 portal[i] = (pl, pr)，最后加一个退化入口 (G, G):
    // 尝试收紧右边
    if pr 在当前右边的左侧（或与右边重合）:
        if apex == right 或 pr 在左边的右侧:        // 收紧后漏斗没有翻转
            right = pr
        else:                                        // 右边越过了左边：左端点成为拐点
            输出 left 作为拐点；apex = left
            从 left 所在的入口之后重新开始扫描
    // 对左边做对称的操作
    ...
输出 G
```

每个入口最多被处理常数次，整体接近 O(n)。拉直之后的拐点通常还要沿多边形的法线方向往里挪一点（Detour 有专门的选项），避免角色紧贴墙角走。

7.3 分帧寻路：别让一次长查询卡一帧

A* 的主循环天然可以暂停：把 open、g、parent 保存在对象里，每帧只展开固定数量的节点，下一帧接着跑。本书的 `Search::step(budget)` 就是这样实现的。

E7：迷宫，左上 → 右下（路径 36968 格）	帧数	单帧最大耗时	总耗时
不分帧	1	**4.312 ms**	4.312 ms
每帧最多展开 20000 个节点	6	0.964 ms	3.842 ms
每帧最多展开 5000 个节点	21	0.248 ms	3.897 ms
每帧最多展开 1000 个节点	103	**0.045 ms**	3.527 ms

总工作量不变（都是 102253 次展开），分帧只是把它摊到多帧里，单帧峰值从 4.3 ms 降到 0.045 ms。工程上通常做成**全局寻路请求队列**：

- **每帧总预算**：比如每帧所有寻路请求加起来最多展开 2 万个节点，或者最多花 1 ms。
- **优先级**：玩家控制的单位、视野内的单位、离目标近的请求优先。
- **请求合并与作废**：同一个单位连续点了三次目标，只保留最后一次；单位死亡时取消它的请求。
- **等待期间的表现**：单位可以先朝目标的直线方向起步，或者播放“思考”动作；分层寻路可以先给出抽象路径的前几段。
- Detour 提供了分帧接口：`initSlicedFindPath` / `updateSlicedFindPath(maxIter)` / `finalizeSlicedFindPath`；`dtPathQueue` 就是一个现成的请求队列。

7.4 动态障碍与重新规划

情况	常见做法
门开关、桥塌、建筑放下	格子图：改格子，更新连通块编号；navmesh：按 tile 局部重建（Recast 的 TileCache 支持用圆柱体 / 盒子作为临时障碍，只重建受影响的 tile）
已规划的路径被挡住	每帧只检查路径前方几个路点是否仍然可走；被挡时从当前位置重新寻路，而不是每帧都重新寻路
其他移动单位	**不要**放进寻路图里（它们一直在动，路径会抖），交给局部避障（7.6 节）
频繁小范围变化	增量搜索算法 LPA* / D* Lite 能复用上次的搜索结果，机器人领域常用；游戏里因为实现复杂、内存大，较少使用

7.5 很多单位去同一个地方：流场

RTS 里一次框选 200 个单位点击地图，跑 200 次 A* 很浪费：目标相同，路径大部分重叠。**流场（flow field）**反过来做：

1. 从目标出发跑一次 Dijkstra（或 BFS），得到每个格子到目标的距离，叫**积分场**；
2. 每个格子指向相邻格子里距离最小的那个，得到**方向场**；
3. 每个单位每帧只需查自己所在格子的方向，O(1)。

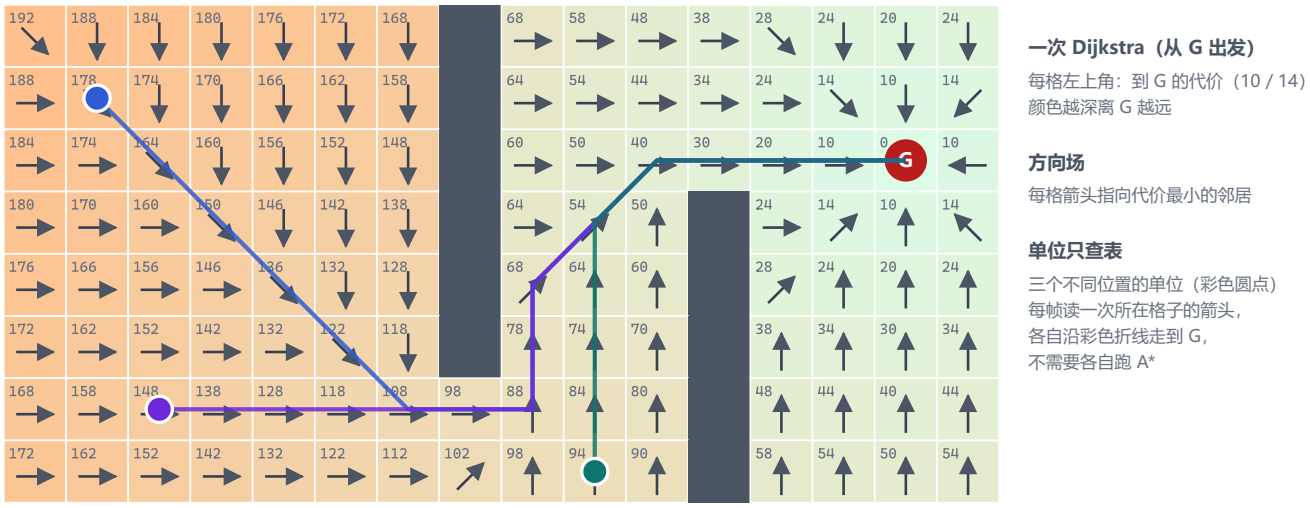


图 7-4 流场：积分场（数字与颜色）+ 方向场（箭头）。由构建前的脚本真实计算（8 方向、禁止切角）

- **代价与单位数量无关**：一次 Dijkstra 的代价摊给所有单位。目标多、单位少时不划算。

- 《最高指挥官 2》采用了按区块（tile）划分、按需计算的流场（Elijah Emerson，Game AI Pro 中有专文介绍），配合局部避障处理大规模单位移动。
- 塔防游戏里所有敌人走向同一个基地，流场几乎是标准做法；玩家放下塔时重算一次。

7.6 全局寻路与局部避障是两层



图 7-3　单位移动的四层结构：A* 只负责最上面一层

面试注意：“两个单位在窄路上互相卡住怎么办”

这不是 A* 的问题，而是局部避障的问题。回答思路：全局寻路不考虑移动单位；局部用 RVO/ORCA 互相让行；仍然卡住时（死锁）设置超时，让一方等待、后退，或者临时把对方当作障碍重新寻路。MOBA 里小兵互相推挤，通常是“沿兵线路点走 + 分离力 / 避让”这一套，而不是每个小兵都跑一次 A*。

7.7 帧同步：寻路必须逐位确定

帧同步（lockstep）游戏的各个客户端只交换玩家输入，各自运行完全相同的逻辑。只要某一端单位走的路和别的端不一样，后面的伤害、技能判定就全部分叉。寻路是最容易出问题的地方之一，因为**等长的路径很多，走哪条取决于平局怎么处理。**

E9 · 两份不同的堆实现（`std::priority_queue` 与自写 `4` 叉堆），同一张图、同一批 `40` 次查询

```
[开阔地]
  只比 f                    路径不同 40 / 40 次，其中代价不同 0 次
```

```
  比 (f, h)                路径不同  0 / 40 次
  全序 (f, h, 格子下标)     路径不同  0 / 40 次
[房间]
  只比 f                   路径不同 39 / 40 次，其中代价不同 0 次
  比 (f, h)                路径不同  0 / 40 次
  全序 (f, h, 格子下标)     路径不同  0 / 40 次
```

只比 f 时，两份实现找到的路径代价完全一样（都是最短的），但走法几乎全不一样。“换一种堆实现”在真实项目里对应的是：**Windows 客户端用 MSVC 的 STL，Android/iOS 用 libc++，服务器用 libstdc++**，三者的堆算法细节都不同。比较 (f, h) 在这批查询里碰巧没有出现差异，但 f 和 h 都相同的节点仍然存在（比如关于起终点连线对称的两个格子），只有加上格子下标形成**全序**，才能从原理上保证任何堆实现给出同一个结果。

帧同步寻路检查清单

1. **代价和启发函数用整数或定点数**：不要用 `float` 和 `sqrt`。不同编译器、指令集、优化选项下，浮点结果可能有末位差异，平局判断就会不同。8 方向用 10 / 14 或 1000 / 1414，欧几里得距离用整数平方根或定点数。
2. **比较器是全序**：(f, h, 节点编号)，任何两个不同节点都能分出先后。
3. **邻居遍历顺序固定**：方向数组写死；navmesh 上多边形邻接表的顺序要来自确定的烘焙数据。
4. **不依赖哈希容器的遍历顺序**：`std::unordered_map` 的遍历顺序取决于实现和插入历史。
5. **导航数据各端相同**：烘焙产物是资源，不能在各端运行时各自生成（浮点体素化在不同机器上可能有差异）。
6. **动态障碍按逻辑帧生效**：障碍的增删必须是逻辑帧里的确定事件，而不是“渲染帧里某个时刻”。
7. **异步 / 分帧的结果在确定的逻辑帧生效**：比如“请求帧 + 2”，而不是“算完就用”。每帧的展开预算也要是逻辑上的固定值，不能按真实耗时（毫秒）分帧。
8. **平滑、视线检测同样要确定**：拉绳用整数格子检测；如果用浮点射线检测，同样存在跨平台差异。

游戏客户端视角：MOBA 里哪些地方会用到寻路

摇杆操作的英雄移动本身不需要寻路，碰到墙沿墙滑动即可。寻路一般出现在：AI 控制的英雄（人机、掉线托管）、召唤物和宠物跟随、野怪脱战回到出生点、点击小地图自动移动、小兵在兵线之外追击目标。它们都在逻辑帧里运行，以上清单全部适用。排查不同步时，可以把每次寻路的输入（起终点、地图版本）和输出路径的哈希写进逻辑日志，第一处不一致的地方往往就能直接定位到问题。

本章速记

- **导航网格**：Recast 烘焙（体素化 → 过滤 → 半径收缩 → 区域 → 凸多边形）；Detour 查询（吸附 → 多边形 A* → 漏斗拉直）。H_SCALE 0.999。
- **漏斗算法**：沿入口序列维护左右两条边，越不过去时端点成为拐点，近似 O(n)。
- **分帧**：每帧展开预算 + 全局请求队列 + 优先级 + 作废（E7：单帧峰值 4.3 ms → 0.045 ms）。
- 动态障碍局部重建、只检查前方路点；移动单位交给局部避障，不进寻路图。
- **流场**：同一目标、大量单位时，一次 Dijkstra 摊给所有单位。
- **帧同步**：整数代价、全序比较、固定邻居顺序、不依赖哈希遍历、确定帧生效（E9：只比 f 时 40 次里 40 次路径不同）。

CHAPTER 08

实测大比拼：五种地图 × 五种算法

同一台机器、同一份代码框架、同一批查询。地图形状对结果的影响，比算法本身还大。

测试环境

Windows 11 x64，MSVC 14.44（`cl /std:c++latest /EHsc /O2`）。512×512 格子图，8 方向，禁止切角，直走 1000、斜走 1414。每张图 40 次查询（U 型陷阱：起点在杯内、终点在杯外；其余：最大连通块内随机取点）。A* 用 Octile 启发、f 相同取 h 小；JPS 为禁止切角版本。“展开”指从 open 取出并处理的节点数，JPS 中指跳点数。源码 `demos/astar_demo.cpp`，原始输出 `demos/run_output.txt`。

8.1 E1 总表

地图（平均最优路径长）	算法	平均展开	平均耗时 ms	代价 / 最优	非最优占比
开阔地 (266.6 格)	BFS	128450	1.758	1.0000	0%
	Dijkstra	127329	5.726	1.0000	0%
	贪心	228	0.033	1.0000	0%
	A*	**228**	**0.018**	1.0000	0%
	JPS	3	0.444	1.0000	0%
随机障碍 25% (282.9 格)	BFS	91515	3.260	1.0081	**72%**
	Dijkstra	90947	6.259	1.0000	0%
	贪心	360	0.043	**1.1444**	**100%**
	A*	11758	1.277	1.0000	0%
	JPS	6285	**0.858**	1.0000	0%
U 型陷阱 (481.7 格)	BFS	205173	2.809	1.0000	0%
	Dijkstra	207016	9.103	1.0000	0%
	贪心	42496	3.054	**1.1055**	**52%**
	A*	52359	3.934	1.0000	0%
	JPS	7	**0.524**	1.0000	0%
迷宫 (19146.2 格)	BFS	80179	2.294	1.0000	0%
	Dijkstra	80180	3.155	1.0000	0%
	贪心	55842	1.527	1.0000	0%
	A*	79476	2.910	1.0000	0%
	JPS	23927	**1.149**	1.0000	0%

房间 (280.0 格)	BFS	114689	1.879	1.0083	**58%**
	Dijkstra	115138	5.872	1.0000	0%
	贪心	382	0.055	**1.1448**	**98%**
	A*	7822	0.920	1.0000	0%
	JPS	564	**0.192**	1.0000	0%

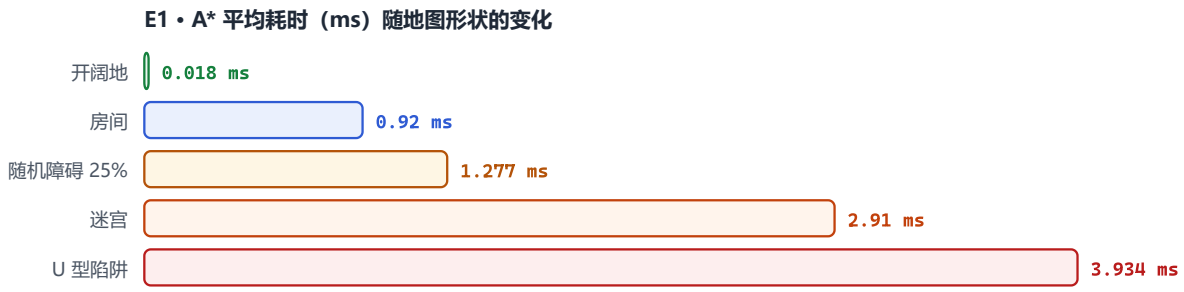


图 8-1 同样 512×512、同样的 A*：耗时相差 200 多倍。决定 A* 快慢的，是启发函数被障碍误导的程度

8.2 从数据中得出的十条结论

1. **A* 的表现取决于地图**：开阔地 0.018 ms，U 型陷阱 3.9 ms，相差 200 多倍。h 被凹形障碍误导时，A* 要先把陷阱填满。
2. **迷宫里 A* ≈ Dijkstra**（79476 对 80180）：真实路径长 19146 格，启发函数估计的直线距离只有几百格，几乎不提供信息；A* 还多了算 h 的开销。
3. **BFS 在 8 方向上不保证最短**：随机图、房间图上 58%～72% 的路径偏长；但它常数小，开阔地比 Dijkstra 快 3 倍。
4. **贪心是一把双刃剑**：开阔地和 A* 一样快，有障碍时路径平均长 10%～14%，没有上界。
5. **JPS 擅长“开阔 + 障碍”**：U 型陷阱快 7.5 倍、房间快 4.8 倍；在纯开阔地上反而比 A* 慢 25 倍（扫描线铺满全图）。
6. **Octile 是 8 方向的正确启发**：比欧几里得少展开 27%～52%，曼哈顿会丢掉最优性（E2）。
7. **加权 A* 收益依地图而定**：随机图 w = 1.2 展开数降到 1/10，路径只长 3%；U 型陷阱几乎没有收益（E3）。
8. **平局决胜在开阔地上至关重要**：228 对 10772，差 47 倍（E4）。

9. **短查询的瓶颈是复位，不是搜索**：每次清零整图慢 90 倍（E5）；open 很小时线性数组最快。
10. **不可达是最坏情况**：20.5 ms 搜完 19.5 万格，而连通块编号的判断只要一次数组比较（E8）。

面试注意：用数据说话要带条件

说“JPS 比 A* 快 4.8 倍”时要加上“在房间类地图、512×512、8 方向禁止切角的条件下”。同样的 JPS 在开阔地上慢 25 倍。面试官更看重你知道“什么情况下快、什么情况下慢、为什么”，而不是一个孤立的倍数。

APPENDIX

附录　面试题库与速记卡

40 道追问，按“基础 → 启发函数 → 正确性与复杂度 → 工程实现 → 加速算法 → 游戏场景”排列。先自己口头回答，再展开看参考答案。

CHAPTER 09

40 道面试追问

每道题的参考答案都压缩到 30 ~ 60 秒能讲完的长度。

9.1 基础（1 ~ 8）

▶ 1. A* 的 f、g、h 分别是什么?

▶ 2. A* 和 Dijkstra、BFS、贪心是什么关系?

▶ 3. open list 和 closed list 是什么?

▶ 4. 为什么要在出队时判断终点，而不是入队时?

▶ 5. 8 方向寻路的代价怎么设？什么是切角?

▶ 6. 游戏里寻路图有哪几种表示？各自优缺点?

▶ 7. 什么时候用 BFS / Dijkstra 而不是 A*?

▶ 8. 如何从 A* 的结果得到路径?

9.2 启发函数（9 ~ 16）

▶ 9. 什么是可采纳？什么是一致?

▶ 10. 4 方向、8 方向、任意角度分别用什么启发?

▶ 11. h 高估会怎样? h = 0 会怎样?

▶ 12. 有地形代价时启发函数怎么设计?

▶ 13. 加权 A* 是什么? 有什么保证?

▶ 14. 什么是平局决胜? 为什么重要?

▶ 15. 两个可采纳的 h 怎么组合?

▶ 16. 改成整数代价后要注意什么?

9.3 正确性与复杂度（17～22）

▶ 17. 证明 A* 的最优性。

▶ 18. 一致性为什么能保证“出队即最优”?

▶ 19. A* 的时间、空间复杂度?

▶ 20. 什么叫 A* 最优效率?

▶ 21. A* 的最坏情况是什么? 怎么应对?

▶ 22. h 只可采纳不一致，会出什么问题?

9.4 工程实现（23～29）

▶ 23. open list 用什么数据结构？decrease-key 怎么做？

▶ 24. 每次寻路都要清空整张图的状态吗？

▶ 25. A* 还有哪些常数优化？

▶ 26. 一帧内有很多寻路请求怎么办？

▶ 27. 寻路怎么做多线程？

▶ 28. A* 写出来路径是锯齿状的怎么办？

▶ 29. 手写 A* 常见 bug？

9.5 加速算法（30～34）

▶ 30. JPS 的原理？

▶ 31. JPS 一定比 A* 快吗？限制是什么？

▶ 32. HPA* 的原理和代价？

▶ 33. 双向 A* 的坑？

▶ 34. ALT 是什么？

9.6 游戏场景（35～40）

▶ 35. 导航网格上的寻路流程？

▶ **36. 漏斗算法的原理?**

▶ **37. 动态障碍怎么处理?**

▶ **38. 200 个单位去同一个地方怎么寻路?**

▶ **39. 两个单位在窄路上互相卡住怎么办?**

▶ **40. 帧同步游戏里寻路要注意什么?**

9.7 速记卡

一页纸背完 A*

- **核心**：f = g + h，展开 open 中 f 最小者；出队时判终点；ng < g 才更新；open 空即不可达。
- **家族**：BFS（等权）、Dijkstra（h = 0）、贪心（只看 h）、加权 A*（≤ w 倍最优）。
- **启发**：可采纳 → 最短；一致 → 出队即最优、不重开。4 向曼哈顿、8 向 Octile、任意角度欧几里得；有地形乘最小单价。
- **证明**：最短路径上总有 f ≤ C* 的节点在 open 里；A* 展开 f < C* 的全部节点，最优效率。
- **复杂度**：O(E log V) 时间、O(V) 空间；最坏是不可达 → 连通块编号预判。
- **工程**：priority_queue + 惰性删除；generation 复位；f 同取 h 小；整数代价；分帧 + 请求队列；多线程状态各自一份。
- **加速**：JPS（均匀格子、开阔 + 障碍）、JPS+、HPA*（分层）、ALT（地标）、双向（注意终止条件）；拉绳 / Theta* / 漏斗做平滑。
- **游戏**：Recast/Detour = 烘焙 + 多边形 A* + 漏斗；流场管大量单位；全局寻路与局部避障分层；帧同步要整数 + 全序 + 确定帧生效。